



①Artificial Intelligence

②Machine Learning

By Jawad Jammoul

Content:

Ai:

Chapter 01 : Introduction to Artificial Intelligence	page 01
Chapter 02 : Intelligent Agents	02
Chapter 03 : Solving problems by searching	06
Chapter 04 : Adversial Search	10
Chapter 05 : First-Order Logic	12
Chapter 06 : Inference in First-Order Logic	22

Discrete Mathematics:

Chapter 01 : The Foundations - Logic and Proofs[1]	30
Chapter 02 : The Foundations - Logic and Proofs[2]	32

Machine Learning:

Chapter 01 : Introduction to Machine Learning	35
Chapter 02 : Data Exploration	47
Chapter 03 : Types of ML & First ML	52
Chapter 04 : Linear Regression algorithm	57
Chapter 05 : Naive Bayes	61
Chapter 06 : KNearestNeighbors(KNN)	67
Chapter 07 : Features Scaling Normalization Standardization	77
Chapter 08 : Decision Tree & Random Forest	83
Chapter 09 : K-Means	91

Chapter 01 : Introduction to Artificial Intelligence

01*What is AI ?

AI is the ability of machines or computer systems to perform tasks that normally require human intelligence , such as learning , reasoning , and decision-making

02*Birth of AI:

A-Classical Period (1950s–1970s):

This is the early stage of AI when researchers tried to make machines follow logical rules and solve problems using strict step-by-step instructions (symbolic AI).

Example: Early programs that solved math problems or proved simple theorems using predefined rules.

B-Romantic Period (1970s–1990s):

During this period, AI research slowed down and expectations became more realistic. Scientists focused on knowledge-based systems and trying to imitate human expertise rather than full intelligence.

Example: Expert systems used in medicine to help doctors diagnose diseases based on stored rules.

C-Modern Period (1990s–present):

AI became powerful due to big data, faster computers, and machine learning techniques that allow systems to learn from data instead of fixed rules.

Example: AI systems used in medical diagnosis, like detecting cancer from MRI scans using deep learning models.

03*Acting Humanly : The Turing Test approach

Means building AI systems that behave in a way that is indistinguishable from human behavior. The goal is not just to solve problems correctly , but to respond communicate , and act like a real person in different situations

The Turing Test , proposed by Alan Turing , is a method to evaluate whether a machine can exhibit human-like intelligence. In this test, if a human cannot reliably distinguish between responses from a machine and a real person during a conversation , the machine is considered intelligent

04*Thinking Humanly : The cognitive modeling approach

Means designing AI systems that try to understand and reproduce how the human mind works. The focus is not only on solving problems, but on simulating human thought processes such as memory, reasoning, learning, and decision-making.

The cognitive modeling approach studies how humans think by observing psychology and brain behavior, then builds AI systems that follow similar mental processes. The goal is to make the computer “think” in a way similar to humans, not just act correctly.

05*Thinking Rationally : The “Laws of thought” approach

Means designing AI systems that make decisions based on logic and correct reasoning rules. The goal is to ensure that the machine reaches a logically valid conclusion, even if it does not think like a human.

The “Laws of thought” approach is based on formal logic principles that define correct reasoning. AI systems using this approach try to represent knowledge in logical statements and use inference rules to derive conclusions that are always logically valid.

06*Acting Rationally : The rational agent approach

Means designing AI systems that choose actions that maximize the chance of achieving the best possible outcome. The system does not need to think like humans, but it must act in a way that is most efficient and beneficial based on the available information.

The rational agent approach defines AI as an “agent” that perceives its environment through sensors and acts on it through actuators, always aiming to achieve the best expected result. It evaluates different actions and chooses the one that maximizes performance or success.

Example: A self-driving car that constantly analyzes road conditions, traffic, and obstacles, then decides the safest and most efficient driving actions in real time.

Chapter 02 : Intelligent Agents

01*What is an Agent?

An **agent** is anything that can perceive its environment through sensors and act upon that environment using actuators to achieve a goal. It takes input from the world, processes it, and produces an output action.

02*Percepts and actions:

Percepts are the inputs that an agent receives from its environment through sensors. They represent what the agent perceives or observes at a specific moment or over time.

Actions are the outputs or operations that an agent performs on its environment through actuators based on its percepts.

03*Agent Program & Agent Function:

The **agent function** is a mathematical description that maps every possible sequence of percepts (what the agent has observed so far) to an action. It defines what the agent should do in every possible situation.

The **agent program** is the actual implementation (code) that runs on a computer to realize the agent function. It takes percepts as input and produces actions as output based on programmed rules or learning methods.

04*Vacuum cleaner agent:

A **vacuum cleaner** agent is a simple AI example that operates in an environment (like a room or grid) and tries to clean dirt based on what it perceives. It uses sensors to detect whether the current location is dirty or clean, and then performs actions like moving or sucking dirt.

05*Rational Agent:

A **rational agent** is an agent that always chooses the action that is expected to achieve the best outcome or maximize its performance measure, given the percepts it has. It does not need to be perfect, but it must act in the most efficient way based on available information.

06*Performance measure:

A **performance measure** is a clear criterion used to evaluate how successful an agent is at achieving its goal in an environment. It defines what “good behavior” means for the agent and is used to judge whether the agent is acting rationally.

07*Rationality:

Rationality is the property of an agent that chooses actions which maximize its expected performance measure based on the percepts it has received and the knowledge it has. A rational agent does the best it can with the information available at the time, even if the outcome is not always perfect.

08*Omniscience, Learning and autonomy:

Omniscience means having complete knowledge about everything and always knowing the correct outcome of every action. In AI, it is impossible in real-world environments because the agent cannot know all future events or hidden information.

Learning is the ability of an agent to improve its performance over time by gaining experience from percepts and feedback. Instead of being fully pre-programmed, a learning agent adapts to new situations.

Autonomy refers to how independently an agent can operate without relying heavily on built-in knowledge from humans. A highly autonomous agent learns from its own experience and makes decisions on its own.

09*Task Environment:

A **task environment** is the external world in which an AI agent operates. It includes everything the agent interacts with, such as inputs, outputs, rules, and conditions that define the problem it must solve.

Specifying the Task Environment:

To design an AI agent properly, we must clearly define its task environment: what the agent observes, what actions it can take, and what goals it must achieve. This helps us understand the problem the agent is solving.

PEAS:

PEAS is a framework used to describe an AI agent's task environment

Performance measure: How success is evaluated

Environment: The world in which the agent operates

Actuators: How the agent acts on the environment

Sensors: How the agent perceives the environment

Properties of Task Environments:

Task environments can have different properties

Fully observable vs partially observable: Does the agent see everything or only part of the environment?

Deterministic vs stochastic: Are outcomes predictable or random?

Episodic vs sequential: Does each action depend on previous actions?

Static vs dynamic: Does the environment change while the agent is thinking?

Discrete vs continuous: Are states/actions limited or continuous?

Single-agent vs multi-agent: Is there only one agent or many interacting agents?

10*The Structure of Agents:

The **agent program** is the software or set of instructions that determines what action the agent should take based on its percepts. It implements the agent function, which maps percept sequences to actions.

Architecture is the physical or hardware platform on which the agent program runs. It provides the sensors and actuators needed for interaction with the environment.

An intelligent agent is created by combining the architecture with the agent program:

AI Agent = Architecture + Agent Program

11*Types of Agent Programs:

1-Simple Reflex Agent:

A **simple reflex agent** acts only based on the current percept, using condition–action rules without considering past information.

- React only to the current percept
- Use condition–action rules
- Do not store past information
- Suitable for simple environments

2-Model-Based Reflex Agent:

A **model-based reflex agent** keeps an internal model of the environment to track aspects that cannot be directly observed. It uses both current percepts and stored information to make decisions.

- Maintain an internal model of the environment
- Use past percepts with current information
- Can handle partially observable environments
- Track changes in the environment over time

3-Goal-Based Agent:

A **goal-based agent** chooses actions by considering future consequences and selecting actions that help achieve a specific goal.

- Act to achieve specific goals
- Consider future consequences of actions
- Compare different possible actions
- More flexible than reflex agents

4-Utility-Based Agent:

A **utility-based agent** selects actions that maximize overall satisfaction or “utility,” especially when several goals or choices are possible.

- Maximize utility or overall satisfaction
- Evaluate multiple possible outcomes
- Handle trade-offs between different goals
- Choose the most beneficial action

12*Learning Agents:

A learning agent is an AI agent that can improve its performance over time by learning from experience and feedback from the environment. Instead of relying only on fixed rules, it adapts its behavior to perform tasks more effectively.

Main Components of a Learning Agent:

- **Learning element:** Improves the agent’s knowledge and behavior
- **Performance element:** Chooses external actions
- **Critic:** Evaluates the agent’s performance and provides feedback
- **Problem generator:** Suggests new actions for learning and exploration

13*Advantages of Simpler Environments:

- Easier to understand and model
- Require less computational power
- Faster decision-making and learning
- Fewer errors due to reduced complexity
- Easier testing and debugging

Chapter 03 : Solving problems by searching

01*Problem Solving Agents:

A problem-solving agent is an AI agent that decides what to do by formulating a problem and then searching for a sequence of actions that leads to a goal state. It uses a systematic approach to move from an initial state to a desired goal state.

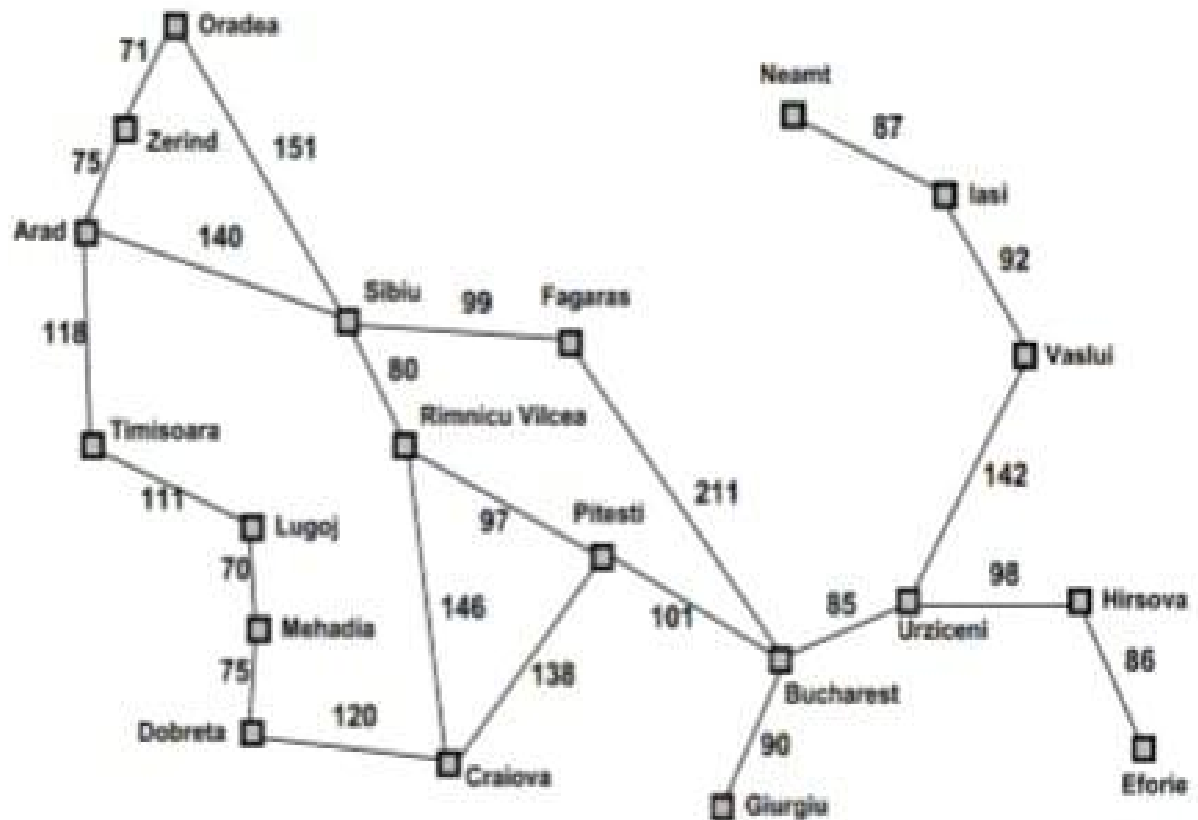
Goal formulation: Defines what it wants to achieve

Problem formulation: Defines states, actions, and the environment

Search: Finds a sequence of actions (solution)

Execution: Performs the actions to reach the goal

Map of Romania:



02*Search Problem Components:

1-Initial State : The starting point of the problem, where the agent begins its search.

2-State Space : The set of all possible states that can be reached from the initial state by applying actions.

3-Actions (Operators) : The possible moves the agent can take from any state.

4-Transition Model (Result Function) : Defines what happens when an action is applied to a state, i.e., it returns the next state.

5-Goal Test : A function that checks whether a given state is the goal state.

6-Path Cost : A numerical value that measures the cost of a path (time, distance, steps, etc.).

03*Formulating Problems:

Abstraction means simplifying the real world by removing unnecessary details and keeping only the information that is relevant for solving the problem. This helps reduce complexity and makes the search process easier and faster.

Why Abstraction is Important?

- Reduces complexity of the problem
- Makes search faster and more efficient
- Focuses only on relevant information
- Helps represent real-world problems in a simplified model

04*State Space Graphs:

A state space graph is a representation of all possible states of a problem and the transitions (actions) between them. It shows how an AI agent can move from the initial state to the goal state.

A node represents a specific state in the problem space. Each node corresponds to a possible configuration of the problem at a certain point.

The goal test is a function applied to a node to check whether it is the goal state or not. It returns true if the node satisfies the goal condition.

05*Search tree:

A **search tree** is a tree representation used in AI search problems to explore possible paths from the initial state to the goal state. It is built during the search process and shows how different actions lead to new states.

Searching for solution:

Leaf Node : A node with no children (no further expansion).

Frontier : All generated but not yet expanded nodes.

Search Strategy : A method used to choose which node to expand next.

Loopy Path : A path that repeats the same state and forms a cycle.

06*Graph Search:

Graph search is a search method that explores a state space while avoiding repeated states by keeping track of explored nodes (closed list) to prevent loops and redundant work.

07*Infrastructure for search algorithms:

Refers to the basic components needed to implement and run any search method in AI.

08*Measuring problem-solving performance:

Problem-solving performance is measured by evaluating how well a search algorithm finds solutions efficiently and correctly.

Completeness : Whether the algorithm always finds a solution if one exists.

Optimality : Whether the algorithm finds the best (lowest cost) solution.

Time Complexity : How much time the algorithm takes to find a solution.

Space Complexity : How much memory the algorithm needs during the search process.

09*Uninformed Search Strategies:

Uninformed search strategies are search methods that do not use any additional knowledge about the goal. They only use the information provided in the problem definition.

1-Breadth-First Search (BFS):

Expands the shallowest nodes first level by level.

2-Depth-First Search (DFS):

Expands the deepest node first and goes as far as possible before backtracking.

3-Uniform-Cost Search (UCS):

Expands the node with the lowest path cost first.

4-Depth-Limited Search (DLS):

DFS with a fixed depth limit to avoid infinite paths.

5-Iterative Deepening Search (IDS):

Repeatedly applies DFS with increasing depth limits.

Uninformed Search (Blind Search)	Informed Search (Heuristic Search)
Uninformed search uses only the information given in the problem definition and does not use any extra knowledge about the goal. Example: BFS, DFS, UCS.	Informed search uses additional knowledge (heuristics) to estimate how close a state is to the goal and guide the search more efficiently. Example: A* search, Greedy best-first search.

10*Comparing Uninformed Search Strategies:

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative-Deeping
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a
Time	$O(b^d)$	$O(b^{1+L} \lceil C*/\epsilon \rceil)$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+L} \lceil C*/\epsilon \rceil)$	$O(bm)$	$O(bl)$	$O(bd)$
Optimal?	Yes ^c	Yes	No	No	Yes ^c

11*Informed Search-Heuristic Function:

$h(n)$ =estimated cost of the cheapest path from the state at node n to a goal state

12*Greedy best-first search:

Greedy best-first search tries to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly. Thus, it evaluates nodes by using just the heuristic function; that is, $f(n)=h(n)$.

13*A* Search:

The most widely known form of best-first search is called A* search(pronounced "A-star search"). It evaluates nodes by combining $g(n)$, the cost to reach the node, and $h(n)$, the cost to get from the node to the goal:

$$f(n)=g(n)+h(n).$$

14*Optimality of A*:

Because $g(n)$ is the actual cost to reach n along the current path, and $f(n)=g(n)+h(n)$, we have as an immediate consequence that $f(n)$ never overestimates the true cost of a solution along the current path through n .

A heuristic $h(n)$ is consistent if, for every node n and every successor n' of n generated by any action a , the estimated cost of reaching the goal from n is no greater than the step cost of getting to n' plus the estimated cost of reaching the goal from n' :

$$h(n) \leq c(n,a,n') + h(n')$$

The proof follows directly from the definition of consistency. Suppose n' is a successor of n , then:

$$g(n') = g(n) + c(n,a,n')$$

$$\text{and we have } f(n') = g(n') + h(n') = g(n) + c(n,a,n') + h(n') \geq g(n) + h(n) = f(n).$$

Chapter 04 : Adversial Search

01*Games definitions:

S0: The initial state, which specifies how the game is set up at the start.

PLAYER(s): Defines which player has the move in a state.

ACTIONS(s): Returns the set of legal moves in a state.

RESULT(s, a): The transition model, which defines the result of a move.

TERMINAL-TEST(s): A terminal test, which is true when the game is over and false otherwise. States where the game has ended are called terminal states.

UTILITY(s, p):

A utility function (also called an objective function or payoff function), defines the final numeric value for a game that ends in terminal state s for a player p .

A zero-sum game is (confusingly) defined as one where the total payoff to all players is the same for every instance of the game.

The initial state, ACTIONS function, and RESULT function define the game tree for the game, a tree where the nodes are game-states and the edges are moves.

02*Optimal Decisions in Games:(Minimax strategy)

MAX must find a strategy, which specifies MAX's move in the initial state, then MAX's moves in the states resulting

from MIN, then MAX's moves in the states resulting from every possible response by MIN to those moves, and so on.

An optimal strategy leads to outcomes at least as good as any other strategy when one is playing an infallible opponent

Each half-moves (MAX move or MIN move) is called a ply.

Given a game tree, the optimal strategy can be determined from the minimax value of each node, which we write as MINIMAX(n).

MINIMAX(s) =

UTILITY(s) if TERMINAL-TEST(s)

$\max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a))$ if PLAYER(s) = MAX

$\min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a))$ if PLAYER(s) = MIN

03*The minimax Algorithm characteristics:

- The **minimax algorithm** characteristics:
- The minimax algorithm computes the minimax decision from the current state.
- The minimax algorithm uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations.
- The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are backed up through the tree as the recursion unwinds.
- In the Trivial Game tree, the algorithm first recurses down to the three bottom left nodes and uses the UTILITY function on them to discover that their values are 3, 12, and 8, respectively. Then it takes the minimum of these values, 3, and returns it as the backed-up value of node B.
- A similar process gives the backed-up values of 2 for C and 2 for D. Finally, we take the maximum of 3, 2, and 2 to get the backed-up value of 3 for the root node.
- The minimax algorithm Performance:
- The minimax algorithm performs a complete depth-first exploration of the game tree.
- If the maximum depth of the tree is m and there are b legal moves at each point, then:
The time complexity of the minimax algorithm is $O(bm)$.
The space complexity is $O(bm)$ for an algorithm that generates all actions at once, or $O(m)$ for an algorithm that generates actions one at a time.
- For real games, of course, the time cost is totally impractical, but this algorithm serves as the basis for the mathematical analysis of games and for more practical algorithms.

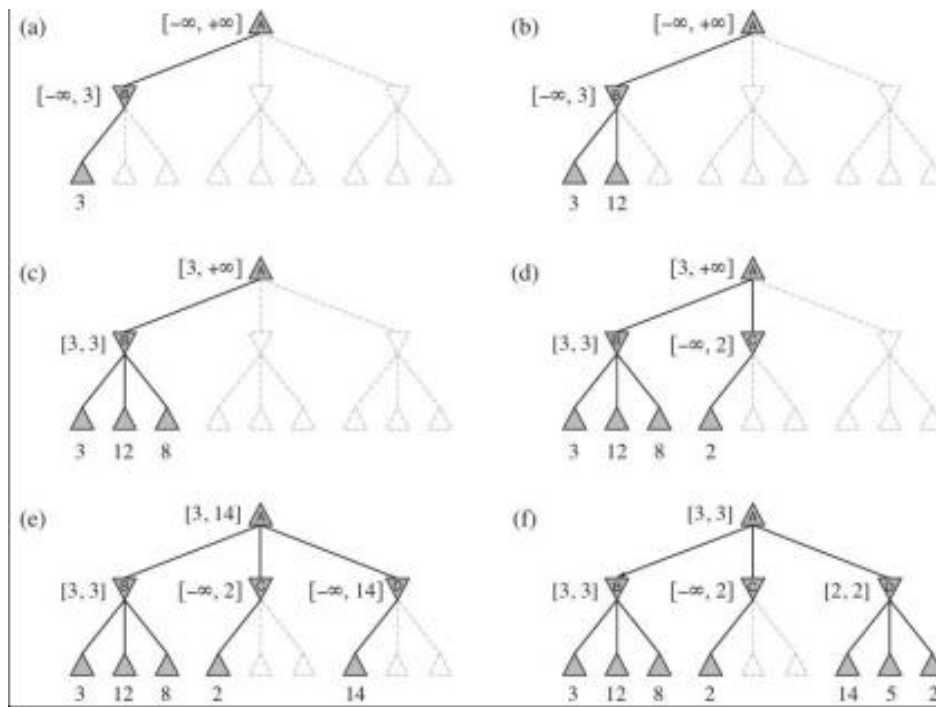
04*Alpha-beta pruning:

- The problem with minimax search is that the number of game states it must examine is exponential in the depth of the tree.
- Unfortunately, we can't eliminate the exponent, but it turns out we can effectively cut it in half.
- The trick is that it is possible to compute the correct minimax decision without looking at every node in the game tree.

Another way to look at this is as a simplification of the formula for MINIMAX.

Let the two un-evaluated successors of node C have values x and y . Then the value of the root node is given by

$$\begin{aligned}\text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2)) \\ &= \max(3, \min(2, x, y), 2) = \max(3, z, 2) \text{ where } z = \min(2, x, y) \leq 2 \\ &= 3.\end{aligned}$$



On a MAX node:

First, assign $v = -\infty$

We check if $v \geq \beta$ then we return v

We assign $\alpha = \text{Max}(\alpha, v)$

On a MIN node:

First, we assign $v = +\infty$

We check if $v \leq \alpha$ then we return v

We assign $\beta = \text{Min}(\beta, v)$

Chapter 05 : First-Order Logic

01*Representation Revisited:

Our discussion motivates the development of first-order logic.

-Programs written in programming languages such as Java or C++ are only computational processes.

-Programming languages variables and data structures can represent facts but lack any general mechanism

for deriving facts from other facts.

-In a programming language we can use a 4×4 array to represent the contents of the wumpus world:

World[2,2] ← Pit is a natural way to assert that there is a pit in square [2,2].

Cannot represent: There is a pit in [2,2] or [3,1]

Cannot represent: If the Wumpus is in [1,1] then he is not in [2,2]

For example, we were forced to write a separate rule about breezes and pits for each square, such as:

$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ "Squares adjacent to pits are breezy."

-The language of thought:

Natural languages (English, Arabic ...) are very expressive, but cannot be used for reasoning systems since it has a lot of rules.

Natural language sentences may reflect multiple meaning to our minds depending on the context.

To store a sentence into a knowledge base, we need to store the context, here a new problem arise: how to store context?

Natural language sentences are ambiguous, which means that the same sentence may reflect different thoughts into our minds.

The famous Sapir-Whorf hypothesis claims that our understanding of the world is strongly influenced by the language we speak. The French have two words "chaise" and "fauteuil," for a concept that English speakers cover with one: "chair." But English speakers can easily recognize the category fauteuil and give it a name, roughly "open-arm chair".

Wanner (1974) did an experiment and found that subjects made the right choice at chance level, about 50% of the time, but remembered the content of what they read with better than 90% accuracy. This suggests that people process the words to form some kind of nonverbal representation. can you remember which of the following two phrases formed the opening of Section 8.1:

"In this section, we discuss the nature of representation languages ..."

"This section covers the topic of knowledge representation languages ..."

Language also seems to influence thought through seemingly arbitrary grammatical features such as the gender of nouns:

For example, "bridge" is masculine in Spanish and feminine in German. Boroditsky (2003) asked subjects to choose English adjectives to describe a photograph of a particular bridge. Spanish speakers chose big, dangerous, strong, and towering, whereas German speakers chose beautiful, elegant, fragile and slender.

Loftus and Palmer (1974) showed experimental subjects a movie of an auto accident. Subjects who were asked "How fast were the cars going when they contacted each other?" reported an average of 32 mph, while subjects who were asked the question with the word "smashed" instead of "contacted" reported 41mph for the same cars in the same movie.

In a first-order logic reasoning system that uses CNF (Conjunctive Normal Form), we can see that the linguistic form " $\neg(A \vee B)$ " and " $\neg A \wedge \neg B$ " are the same because we can look inside the system and see that the two sentences are stored as the same canonical CNF form. Can we do that with the human brain? Until recently the answer was "no", now it is maybe.

Combining the best of formal and natural languages :

We can adopt the foundation of propositional logic, a declarative, compositional semantics that is context-independent and unambiguous and build a more expressive logic on that foundation, borrowing representational ideas from natural language while avoiding its drawbacks.

When we look at the syntax of natural language, the most obvious elements are nouns and noun phrases that refer to objects (squares, pits, wumpuses) and verbs and verb phrases that refer to relations among objects (is breezy, is adjacent to, shoots). Some of these relations are functions, relations in which there is only one "value" for a given "input."

It is easy to start listing examples of objects, relations, and functions:

Objects: people, houses, numbers, theories, Ronald McDonald, colors, baseball games, wars, centuries ...

Relations:

these can be unary relations or properties such as red, round, bogus, prime, multistoried ..., or more general n-ary relations such as brother of, bigger than, inside, part of, has color, occurred after, owns, comes between, ...

Functions:

father of, best friend, third inning of, one more than, beginning of ...

First Order Logic Ontological Commitment: Facts, Objects and Relations

First Order Logic Epistemological Commitment: true/false/unknown

02*Syntax and Semantics of first-order-logic:

-First-Order Logic-Model:

Models of a logical language are the formal structures that constitute the possible worlds under consideration.

-Each model links the vocabulary of the logical sentences to elements of the possible world, so that the truth of any sentence can be determined.

-Models for first-order logic have objects in them:

The domain of a model is the set of objects or domain elements it contains.

The domain is required to be non-empty.

Every possible world must contain at least one object.

-Objects:

Richard the Lionheart, King of England from 1189 to 1199;

His younger brother, the evil King John, who ruled from 1199 to 1215;

The left leg of Richard

The left leg of John

A crown.

Relations:

Brotherhood Relation:

{<Richard the Lionheart, King John>, <King John, Richard the Lionheart> }

On Head Relation:

{<the crown, King John>}

The “brotherhood” and “on head” relations are binary relations, that is, they relate pairs of objects.

Unary Relations or Properties:

Person: {<Richard>, <John>}

King: {<John>}

Crown: {<crown>}

Functions:

Left Leg: Richard the Lionheart → Richard’s left leg

King John → John’s left leg

-First-Order Logic-Symbols and interpretations

Symbols:

Constant Symbols: which stand for objects.

Predicate Symbols: which stand for relations.

Function Symbols: which stand for functions.

We adopt the convention that these symbols will begin with uppercase letters.

Each predicate and function symbol comes with an arity that fixes the number of arguments

Interpretation (Intended Interpretation):

An interpretation maps constant symbols to objects, predicate symbols to relations on those objects, and function symbols to functions on those objects.

Sample Model Interpretation:

Richard refers to Richard the Lionheart and John refers to the evil King John.

Brother refers to the brotherhood relation.

OnHead refers to the “on head” relation that holds between the crown and King John.

Person, King, and Crown refer to the sets of objects that are persons, kings, and crowns.

LeftLeg refers to the “left leg” function.

Notice that not all the objects need have a name, for example, the intended interpretation does not name the crown or the legs.

-First-Order Logic - Terms:

A term is a logical expression that refers to an object.

Constant symbols are therefore terms, but it is not always convenient to have a distinct symbol to name every object.

A complex term is formed by a function symbol followed by a parenthesized list of terms as arguments to the function symbol.

It is important to remember that a complex term is just a complicated kind of name.

It is not a "subroutine call" that "returns a value."

Consider a term $f(t_1, \dots, t_n)$:

The function symbol f refers to some function in the model (call it F)

The argument terms refer to objects in the domain (call them $d_1, \dots, d_n$).

The term refers to the object that is the value of the function F applied to $d_1, \dots, d_n$.

For example, $\text{LeftLeg}(\text{John})$ refers to King John's left leg.

-First-Order Logic - Sentences:

Atomic Sentence: is formed from a predicate symbol optionally followed by a parenthesized list of terms, such as:

$\text{Brother}(\text{Richard}, \text{John})$

This states, under the intended interpretation given earlier, that Richard the Lion heart is the brother of King John.

Atomic sentences can have complex terms as arguments. Thus:

$\text{Married}(\text{Father}(\text{Richard}), \text{Mother}(\text{John}))$

States that Richard the Lionheart's father is married to King John's mother (again, under a suitable interpretation).

An atomic sentence is true in each model if the relation referred to by the predicate symbol holds among the objects referred to by the arguments.

Complex Sentence: is constructed by applying logical connectives to construct more complex sentences, with

the same syntax and semantics as in propositional logic. Here are four sentences that are true in the sample

model under our intended interpretation:

$\neg \text{Brother}(\text{LeftLeg}(\text{Richard}), \text{John})$

$\text{Brother}(\text{Richard}, \text{John}) \wedge \text{Brother}(\text{John}, \text{Richard})$

$\text{King}(\text{Richard}) \vee \text{King}(\text{John})$

$\neg \text{King}(\text{Richard}) \Rightarrow \text{King}(\text{John})$.

-First-Order Logic - Quantifiers:

Once we have a logic that allows objects, it is natural to express properties of entire collections of objects, instead of enumerating the objects by name.

First order logic has two types of quantifiers:

Universal Quantification ($\forall$)

Existential quantification ($\exists$)

-Universal Quantification $\forall$:

$\forall$ is usually pronounced "For all ...".

Intuitively, the sentence $\forall xP$, where P is any logical expression, says that P is true for every object x . Where x is any possible object in the model.

By asserting the universally quantified sentence, which is equivalent to asserting a whole list of individual implications, we end up asserting the conclusion of the rule just for those objects for whom the premise (Left Side of the $\Rightarrow$) is true and saying nothing at all about those individuals for whom the premise is false.

"All kings are persons" is written in first-order logic as:

$\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$.

Thus, the sentence says, "For all x , if x is a king, then x is a person."

The symbol x is called a variable. By convention, variables are lowercase letters. A variable is a term all by itself, and as such can also

serve as the argument of a function, for example, $\text{LeftLeg}(x)$. A term with no variables is called a ground term.

This equivalent to the following assertions:

Richard the Lionheart is a king $\Rightarrow$ Richard the Lionheart is a person

King John is a king $\Rightarrow$ King John is a person

Richard's left leg is a king $\Rightarrow$ Richard's left leg is a person

John's left leg is a king $\Rightarrow$ John's left leg is a person

The crown is a king $\Rightarrow$ the crown is a person.

A common mistake, made frequently even by diligent readers who have read this paragraph several times, is to use conjunction instead of implication.

The sentence:

$\forall x \text{ King}(x) \wedge \text{Person}(x)$ would be equivalent to asserting:

Richard the Lionheart is a king $\wedge$ Richard the Lionheart is a person

King John is a king $\wedge$ King John is a person

Richard's left leg is a king $\wedge$ Richard's left leg is a person and so on.

Obviously, this does not capture what we want.

Nested Quantification:

We will often want to express more complex sentences using multiple quantifiers. The simplest case is where the

quantifiers are of the same type. For example, "Brothers are siblings" can be written as:

$\forall x \forall y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$

Consecutive quantifiers of the same type can be written as one quantifier with several variables. For example, to say

that siblinghood is a symmetric relationship, we can write:

$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow \text{Sibling}(y, x)$

In other cases, we will have mixtures. "Everybody loves somebody" means that for every person, there is someone that

person loves: $\forall x \exists y \text{ Loves}(x, y)$

On the other hand, to say "There is someone who is loved by everyone," we write: $\exists y \forall x \text{ Loves}(x, y)$

The order of quantification is therefore very important. It becomes clearer if we insert parentheses.

$\forall x (\exists y \text{ Loves}(x, y))$

says that everyone has a particular property, namely, the property that they love someone. On the other hand, $\exists y$

$(\forall x \text{ Loves}(x, y))$ says that someone in the world has a particular property, namely the property of being loved by everybody.

Some confusion can arise when two quantifiers are used with the same variable name. Consider the sentence $\forall x (\text{Crown}(x) \vee (\exists x \text{ Brother}(\text{Richard}, x)))$. To avoid the confusion between x within $\text{Crown}(x)$ and x within $\text{Brother}(\text{Richard}, x)$, it is better to use another variable: $\forall x (\text{Crown}(x) \vee (\exists z \text{ Brother}(\text{Richard}, z)))$.

The rule is that the variable belongs to the innermost quantifier that mentions it, then it will not be subject to any other quantification.

-Connection between $\forall$ and $\exists$:

$$\begin{aligned} \forall_x \neg P &\equiv \neg \exists_x P & \neg(P \vee Q) &\equiv \neg P \wedge \neg Q \\ \neg \forall_x P &\equiv \exists_x \neg P & \neg(P \wedge Q) &\equiv \neg P \vee \neg Q \\ \forall_x P &\equiv \neg \exists_x \neg P & P \wedge Q &\equiv \neg(\neg P \vee \neg Q) \\ \exists_x P &\equiv \neg \forall_x \neg P & P \vee Q &\equiv \neg(\neg P \wedge \neg Q) \end{aligned}$$

De Morgan Rules

The two quantifiers are intimately connected with each other, through negation.

Asserting that everyone dislikes parsnips is the same as asserting there does not exist someone who likes them, and vice versa:

$\forall x \neg \text{Likes}(x, \text{Parsnips})$ is equivalent to $\neg \exists x \text{ Likes}(x, \text{Parsnips})$.

We can go one step further: "Everyone likes ice cream" means that there is no one who does not like ice cream:

$\forall x \text{ Likes}(x, \text{IceCream})$ is equivalent to $\neg \exists x \neg \text{Likes}(x, \text{IceCream})$.

Because $\forall$ is really a conjunction over the universe of objects and $\exists$ is a disjunction, it should not be surprising that they obey

De Morgan's rules

Thus, we do not really need both $\forall$ and $\exists$, just as we do not really need both $\wedge$ and $\vee$. Still, readability is more important, so we will keep both quantifiers.

03*First-Order Logic-Equality:

We can use the equality symbol to signify that two terms refer to the same object.

For example, Father (John) = Henry says that the object referred to by Father (John) and the object referred to by Henry are the same.

Because an interpretation fixes the referent of any term, determining the truth of an equality sentence is simply a matter of seeing that the referents of the two terms are the same object.

The equality can also be used with negation to insist that two terms are not the same object.

To say that Richard has at least two brothers, we would write $\exists x, y \text{ Brother}(x, \text{Richard}) \wedge \text{Brother}(y, \text{Richard}) \wedge \neg(x=y)$.

The sentence $\exists x, y \text{ Brother}(x, \text{Richard}) \wedge \text{Brother}(y, \text{Richard})$ does not have the intended meaning.

It is true in the sample model, where Richard has only one brother. To see this, consider the extended interpretation in which both x and y are assigned to King John. The addition of $\neg(x=y)$ rules out such models.

04*The kinship domain:

Knowledge Representation (Knowledge Base): The set of information we know about our abstracted model.

Sentences are added to a knowledge base using TELL, such sentences are called assertions.

For example, we can assert that John is a king, Richard is a person, and all kings are persons:

TELL(KB, King(John))

TELL(KB, Person(Richard))

TELL(KB, $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$)

We can ask questions of the knowledge base using ASK. For example: ASK(KB, King(John))

Questions asked with ASK are called queries or goals.

We can ask quantified queries, such as ASK(KB, $\exists x \text{ Person}(x)$), the result is true. If we want to know what value of x

makes the sentence true, we will need a different function, ASKVARs, which we call with:

ASKVARs(KB, Person(x)) and which yields a stream of answers. In this case there will be two answers: { x /John} and { x /Richard}.

Such an answer is called a substitution or binding list.

05*First-Order Logic:

This domain defines the relationships between persons.

Clearly, the objects in our domain are people.

We have two unary predicates, Male and Female.

Kinship relations, parenthood, brotherhood, marriage, and so on, are represented by binary predicates: Parent, Sibling, Brother, Sister, Child, Daughter, Son, Spouse, Wife, Husband, Grandparent, Grandchild, Cousin, Aunt and Uncle.

We use functions for Mother and Father, because every person has exactly one of each of these (at least according to nature's design).

one's mother is one's female parent:

$$\forall m, c \text{ Mother}(c) = m \Leftrightarrow \text{Female}(m) \wedge \text{Parent}(m, c)$$

One's husband is one's male spouse:

$$\forall w, h \text{ Husband}(h, w) \Leftrightarrow \text{Male}(h) \wedge \text{Spouse}(h, w)$$

Male and female are disjoint categories:

$$\forall x \text{ Male}(x) \Leftrightarrow \neg \text{Female}(x)$$

Parent and child are inverse relations:

$$\forall p, c \text{ Parent}(p, c) \Leftrightarrow \text{Child}(c, p)$$

A grandparent is a parent of one's parent:

$$\forall g, c \text{ Grandparent}(g, c) \Leftrightarrow \exists p \text{ Parent}(g, p) \wedge \text{Parent}(p, c)$$

A sibling is another child of one's parents:

$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow x \neq y \wedge \exists p \text{ Parent}(p, x) \wedge \text{Parent}(p, y)$$

06*The natural numbers:

We describe here the theory of natural numbers or non-negative integers.

We need a predicate NatNum that will be true of natural numbers.

We need one constant symbol 0.

We need one function symbol S (successor). $S(0) = 1$, $S(1) = 2 \dots$

Natural numbers are defined recursively:

$$\text{NatNum}(0)$$

$$\forall n \text{ NatNum}(n) \Rightarrow \text{NatNum}(S(n))$$

That is, 0 is a natural number, and for every object n, if n is a natural number, then S(n) is a natural number.

So, the natural numbers are 0, S(0), S(S(0)), and so on

We also need axioms to constrain the successor function:

$$\forall n \ 0 \neq S(n)$$

$$\forall m, n \ m = n \Rightarrow S(m) = S(n)$$

Now we can define addition in terms of the successor function:

$$\forall m \text{ NatNum}(m) \Rightarrow +(0, m) = m$$

$$\forall m, n \text{ NatNum}(m) \wedge \text{NatNum}(n) \Rightarrow +(S(m), n) = S(+(m, n))$$

The first of these axioms says that adding 0 to any natural number m gives m itself.

The second axiom reduces addition to repeated application of the successor function.

Simple Example: $+(2, 4) = +(S(1), 4) = S(+(1, 4)) = S(+(S(0), 4)) = S(S(+(0, 4))) = S(S(4)) = S(5) = 6$

07*Sets:

Constants, Predicates and Functions:

The empty set is a constant written as $\{\}$

There is one unary predicate, Set, which is true of sets

The binary predicate $x \in s$ (x is a member of set s)

The binary predicate $s1 \subseteq s2$ (set $s1$ is a subset of set $s2$)

The binary function are $s1 \cap s2$ (the intersection of two sets)

The binary function $s1 \cup s2$ (the union of two sets)

The binary function $\{x|s\}$ (the set resulting from adjoining element x to set s)

Axioms:

The only sets are the empty set and those made by adjoining something to a set:

$$\forall s \text{ Set}(s) \Leftrightarrow (s = \{\}) \vee (\exists x, s2 \text{ Set}(s2) \wedge s = \{x|s2\})$$

The empty set has no elements adjoining into it. In other words, there is no way to decompose $\{\}$ into a smaller set

and an element: $\neg \exists x, s \{x|s\} = \{\}$

Adjoining an element already in the set has no effect:

$$\forall x, s \quad x \in s \Leftrightarrow s = \{x|s\}$$

The only members of a set are the elements that were adjoining into it. We express this recursively, saying that x is a member of s if and only if s is equal to some set $s2$ adjoining with some element y , where either y is the same as x or x is a member of $s2$:

$$\forall x, s \quad x \in s \Leftrightarrow \exists y, s2 (s = \{y|s2\} \wedge (x = y \vee x \in s2))$$

A set is a subset of another set if and only if all the first set's members are members of the second set:

$$\forall s1, s2 \quad s1 \subseteq s2 \Leftrightarrow (\forall x \quad x \in s1 \Rightarrow x \in s2)$$

Two sets are equal if and only if each is a subset of the other:

$$\forall s1, s2 \quad (s1 = s2) \Leftrightarrow (s1 \subseteq s2 \wedge s2 \subseteq s1)$$

An object is in the intersection of two sets if and only if it is a member of both sets:

$$\forall x, s1, s2 \quad x \in (s1 \cap s2) \Leftrightarrow (x \in s1 \wedge x \in s2)$$

An object is in the union of two sets if and only if it is a member of either set:

$$\forall x, s1, s2 \quad x \in (s1 \cup s2) \Leftrightarrow (x \in s1 \vee x \in s2)$$

08*Lists:

We can use the vocabulary of Lisp for lists:

Nil is the constant list with no elements

Cons (Construct), Append, First and Rest are functions

-Find is the predicate that does for lists what Member does for sets.

-List? is a predicate that is true only of lists.

-As with sets, it is common to use syntactic sugar in logical sentences involving lists.

The empty list is $[\]$.

-The term $\text{Cons}(x, y)$, where y is a nonempty list, is written $[x|y]$.

-The term $\text{Cons}(x, \text{Nil})$ (i.e., the list containing the element x) is written as $[x]$.

-A list of several elements, such as $[A, B, C]$, corresponds to the nested term $\text{Cons}(A, \text{Cons}(B, \text{Cons}(C, \text{Nil})))$.

Chapter 06 : Inference in First-Order Logic

01*Propositional vs First-Order Interface:

-Inference rules for quantifiers - Universal instantiation:

Consider the following sentence:

$$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$$

Then it seems quite permissible to infer any of the following sentences:

$$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$$

$$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$$

$$\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John}))$$

The rule of Universal Instantiation (UI for short) says that we can infer any sentence obtained by substituting a ground

term (a term without variables) for the variable. To write out the inference rule formally, we use the notion of substitutions, Let $\text{SUBST}(\theta, \alpha)$ denote the result of applying the substitution θ to the sentence α . Then the rule is written:

for any variable v and ground term g . For example, the three sentences given earlier are obtained with the substitutions $\{x/\text{John}\}$, $\{x/\text{Richard}\}$, and $\{x/\text{Father}(\text{John})\}$

-Inference rules for quantifiers - Existential Instantiation ($\exists$):

In the rule for Existential Instantiation, the variable is replaced by a single new constant symbol. The formal statement is as follows: for any sentence α , variable v , and constant symbol k that does not appear elsewhere in the knowledge base :

For example, from the sentence

$$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

we can infer the sentence

$$\text{Crown}(C1) \wedge \text{OnHead}(C1, \text{John})$$

as long as $C1$ does not appear elsewhere in the knowledge base. Basically, the existential sentence says there is some object satisfying a condition, and applying the existential instantiation rule just gives a name to that object. Of course, that name must not already belong to another object. In logic, the new name is called a Skolem constant ($d(x y)/dy = x y$ but not $\top$)

-Reduction to propositional inference:

Once we have rules for inferring non-quantified sentences from quantified sentences, it becomes possible to reduce first-order inference to propositional inference.

The first idea is that, just as an existentially quantified sentence can be replaced by one instantiation, a universally quantified

sentence can be replaced by the set of all possible instantiations.

For example, suppose our knowledge base contains just the sentences

$$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$$

$$\text{King}(\text{John})$$

$$\text{Greedy}(\text{John})$$

$$\text{Brother}(\text{Richard}, \text{John})$$

Then we apply UI to the first sentence using all possible ground-term substitutions from the vocabulary of the knowledge

base—in this case, $\{x/\text{John}\}$ and $\{x/\text{Richard}\}$. We obtain

$$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$$

$$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$$

and we discard the universally quantified sentence. Now, the knowledge base is essentially propositional. This concept is called **propositionalization**

-Entailed sentences:

In First-Order Logic (FOL), a sentence (or formula) α is said to be entailed by a knowledge base (KB) if and only if α is true in every model (interpretation) in which the KB is true. This relationship is denoted by $KB \models \alpha$, which reads as "KB entails α ".

We have sketched an approach to first-order inference via propositionalization that is complete, that is, any entailed sentence can be proved.

The question of entailment for first-order logic is semi-decidable, that is, algorithms exist that say yes to every entailed sentence, but no algorithm exists that also says no to every non-entailed sentence. In other words, if KB entails α then we can find a proof, but in case KB does not entail α , we cannot provide a proof. Since our proof procedure can go on and on, generating more and more deeply nested terms, but we will not know whether it is stuck in a hopeless loop or whether the proof is just about to pop out.

02*Unification and lifting:

-Lifting:

Suppose we have the following sentences:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John}) \wedge \forall y \text{ Greedy}(y)$

How to proof Evil(John)?

if there is some substitution θ that makes each of the conjuncts of the premise of the implication identical to sentences already in the knowledge base, then we can assert the conclusion of the implication, after applying θ .

This inference process can be captured as a single inference rule that we call Generalized Modus Ponens: For atomic sentences p_i , p_i' , and q , where there is a substitution θ such that $\text{SUBST}(\theta, p_i') = \text{SUBST}(\theta, p_i)$, for all i :

There are $n+1$ premises to this rule: the n atomic sentences p_i' and the one implication. The conclusion is the result of applying the substitution θ to the consequent q . For our example:

p_1' is $\text{King}(\text{John})$

p_1 is $\text{King}(x)$

p_2' is $\text{Greedy}(\text{John})$

p_2 is $\text{Greedy}(x)$

θ is $\{x/\text{John}, y/\text{John}\}$

q is $\text{Evil}(x)$

$\text{SUBST}(\theta, q)$ is $\text{Evil}(\text{John})$.

Generalized Modus Ponens is a lifted version of Modus Ponens, it raises Modus Ponens from ground (variable-free) propositional logic to first-order logic.

-Unification:

Lifted inference rules require finding substitutions that make different logical expressions look identical. This process is called unification and is a key component of all first-order inference algorithms. The UNIFY algorithm takes two sentences and returns a unifier for them if one exists:

$\text{UNIFY}(p, q) = \theta$ where $\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$

Let us look at some examples of how UNIFY should behave. Suppose we have a query

$\text{AskVars}(\text{Knows}(\text{John}, x))$: whom does John know?

Answers to this query can be found by finding all sentences in the knowledge base that unify with $\text{Knows}(\text{John}, x)$.

Here are the results of unification with four different sentences that might be in the knowledge base:

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(\text{John}, \text{Jane})) = \{x/\text{Jane}\}$

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Bill})) = \{x/\text{Bill}, y/\text{John}\}$

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, \text{Mother}(y))) = \{y/\text{John}, x/\text{Mother}(\text{John})\}$

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x, \text{Elizabeth})) = \text{fail}$.

The last unification fails because x cannot take on the values John and Elizabeth at the same time.

Now, remember that

$\text{Knows}(x, \text{Elizabeth})$ means "Everyone knows Elizabeth," so we should be able to infer that John knows Elizabeth.

The problem arises only because the two sentences happen to use the same variable name, x . The problem can be avoided by

standardizing apart one of the two sentences being unified, which means renaming its variables to avoid name clashes. For

example, we can rename x in $\text{Knows}(x, \text{Elizabeth})$ to $x17$ (a new variable name) without changing its meaning. Now the

unification will work: $\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x17, \text{Elizabeth})) = \{x/\text{Elizabeth}, x17/\text{John}\}$

-Storage and Retrieval:

Underlying the TELL and ASK functions used to inform and interrogate a knowledge base are the more primitive STORE and FETCH functions.

STORE(s) stores a sentence s into the knowledge base and FETCH(q) returns all unifiers such that the query q unifies with some sentence in the knowledge base.

The problem we used to illustrate unification, finding all facts that unify with $\text{Knows}(\text{John}, x)$ —is an instance of FETCHing.

The simplest way to implement STORE and FETCH is to keep all the facts in one long list and unify each query against every element of the list.

Such a process is inefficient, but it works, and it's all you need to understand the rest of the chapter.

03*First-Order Some definitions:

In First-Order Logic (FOL), a term is an expression that represents an object in the domain of discourse.

Terms can be constants, variables,

or function applications. Terms are used to construct more complex logical formulas and are fundamental building blocks in FOL.

In First-Order Logic (FOL), a ground term is a term that does not contain any variables. Ground terms are composed exclusively of constants and function symbols applied to other ground terms. They are fully specified and represent concrete objects or entities in the domain of discourse.

Literal: A literal is an atomic proposition (or variable) or its negation. For example, P and $\neg P$ are literals.

In logic, a clause is a disjunction of literals. It is a basic building block used in various forms of logical expressions, particularly in Conjunctive Normal Form (CNF) and Disjunctive Normal Form (DNF).

Clauses are central to the resolution method in automated theorem proving.

Disjunction: A disjunction is a logical "or" operation. For example, $P \vee Q$ is a disjunction of the literals P and Q .

Conjunction: A conjunction is a logical "and" operation. For example, $P \wedge Q$ is a conjunction of the literals P and Q .

In the context of logic and automated reasoning, the term "sound" refers to the correctness of an inference method. Specifically, an inference method is considered sound if it only produces conclusions that are logically entailed by the premises. In other words, if a method is sound, it will not infer any false conclusions from true premises.

In the context of logic and automated reasoning, the term "complete" refers to the ability of an inference method to derive all possible conclusions that logically follow from the given premises (facts and rules). Specifically, an inference method is considered complete if it can find a solution or derive a conclusion whenever one exists, given enough time and resources.

04*Horn clauses:

A Horn clause is a special type of clause in first-order logic (FOL) that has at most one positive literal. Horn clauses are particularly significant in logic programming and automated reasoning because they have desirable computational properties and are used in systems like Prolog.

Structure of Horn Clauses:

In first-order logic, a clause is typically a disjunction of literals (a literal is either an atomic formula or its negation). A

Horn clause can be defined as follows:

Definite Clause:

A Horn clause with exactly one positive literal. It has the form:

$$L1 \vee \neg L2 \vee \neg L3 \vee \dots \vee \neg Ln$$

which can be written equivalently as

$$L1 \leftarrow L2 \wedge L3 \wedge \dots \wedge Ln$$

Here, $L1$ is the positive literal, and $L2, L3, \dots, Ln$ are negative literals (negated).

Fact:

A Horn clause with no negative literals. It is simply a single positive literal:

$L1$ which can be interpreted as $L1 \leftarrow \text{true}$.

Goal Clause:

A Horn clause with no positive literals. It has the form:

$$\neg L1 \vee \neg L2 \vee \dots \vee \neg Ln$$

which can be written equivalently as:

$$\leftarrow L1 \wedge L2 \wedge \dots \wedge Ln$$

05*Forward chaining:

Every definite clause can be written as an implication whose premise is a conjunction of positive literals and whose conclusion is a single positive literal.

For example, the definite clause

$$(\neg L1 \vee \neg L2 \vee L3)$$

can be written as the implication

$$(L1 \wedge L2) \Rightarrow L3.$$

In the implication form, the sentence is easier to understand.

In Horn form, the premise is called the body and the conclusion is called the head.

The forward-chaining algorithm determines if a single proposition symbol q , the query, is entailed by a knowledge base of definite clauses.

It begins from known facts (positive literals) in the knowledge base.

If all the premises of an implication are known, then its conclusion is added to the set of known facts.

For example, if $L1,1$ and Breeze are known and $(L1,1 \wedge \text{Breeze}) \Rightarrow B1,1$ is in the knowledge base, then $B1,1$ can be added.

This process continues until the query q is added or until no further inferences can be made.

It is easy to see that forward chaining is sound: every inference is essentially an application of Modus Ponens.

Forward chaining is also complete: every entailed atomic sentence will be derived.

06*First-Order definite clauses:

First-order definite clauses closely resemble propositional definite clauses: they are disjunctions of literals of which exactly, one is positive.

A definite clause either is atomic or is an implication whose antecedent is a conjunction of positive literals and whose consequent is a single positive literal.

The following are first-order definite clauses:

$$\text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$$

$$\text{King}(\text{John})$$

$$\text{Greedy}(y).$$

Unlike propositional literals, first-order literals can include variables, in which case those variables are assumed to be universally quantified. (Typically, we omit universal quantifiers when writing definite clauses.) Not every knowledge base can be converted into a set of definite clauses because of the single-positive-literal restriction, but many can.

07*First-Order Forward chaining:

The first forward-chaining algorithm we consider is a simple one.

Starting from the known facts, it triggers all the rules whose premises are satisfied, adding their conclusions to the known facts.

The process repeats until the query is answered (assuming that just one answer is required) or no new facts are added.

First, it is sound, because every inference is just an application of Generalized Modus Ponens, which is sound.

Second, it is complete for definite clause knowledge bases, that is, it answers every query whose answers are entailed by any knowledge base of definite clauses.

08*First-Order Logic conjunctive normal form(CNF):

A first-order logic sentence is in **CNF** if it is a conjunction of clauses where each clause is a disjunction of literals. Literals can contain variables, which are assumed to be universally quantified. For example, the sentence:

$$\forall x \text{ American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$$

Becomes in CNF:

$$\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$$

Every sentence of first-order logic can be converted into an inferentially equivalent CNF sentence.

In particular, the CNF sentence will be unsatisfiable just when the original sentence is unsatisfiable.

This leads to a basis for doing proofs by contradiction on the CNF sentences.

09*First-Order Logic conversion to CNF:

1.Eliminate Implications and Biconditionals:

Replace implications $P \Rightarrow Q$ with $\neg P \vee Q$.

Replace biconditionals $P \Leftrightarrow Q$ with $(\neg P \vee Q) \wedge (\neg Q \vee P)$.

2.Move Negations Inward (Negation Normal Form):

Use De Morgan's laws to move negations inward, eliminating double negations:

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x)$$

$$\neg(\exists x P(x)) \equiv \forall x \neg P(x)$$

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

3.Standardize Variables:

Ensure that each quantifier uses a unique variable to avoid confusion during Skolemization.

4.Skolemization:

Eliminate existential quantifiers by introducing Skolem constants (for ground terms) or Skolem functions (for terms with variables):

If no universal quantifier precedes the existential quantifier, replace the variable with a constant symbol.

$$\exists y P(x, y) \rightarrow P(x, A)$$

Where A is a Skolem Constant

If a universal quantifier precedes the existential quantifier, replace the variable with a function of the universal quantifier variable:

$$\forall x \exists y P(x, y) \rightarrow \forall x P(x, f(x))$$

The Skolem function $f(x)$ is used to replace y in the context of the universally quantified variable x .

5.Drop Universal Quantifiers:

Once all existential quantifiers have been eliminated, drop the universal quantifiers for the remaining variables, assuming they are universally quantified.

6.Distribute $\wedge$ over $\vee$ (Distributive Laws):

Apply the distributive laws to move the formula into CNF:

$$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$$

$$(P \vee Q) \wedge R \equiv (P \wedge R) \vee (Q \wedge R)$$

10*Resolution Inference rule:

Resolution:

is a Method using clauses to derive new clauses and prove satisfiability or un-satisfiability.

Propositional literals are complementary if one is the negation of the other.

First-order literals are complementary if one unifies with the negation of the other.

Resolution Inference Rule:

where $\text{UNIFY}(\ell_i, \neg m_j) = \theta$.

For example, we can resolve the two clauses

$[\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)]$ and $[\neg \text{Loves}(u, v) \vee \neg \text{Kills}(u, v)]$

by eliminating the complementary literals $\text{Loves}(G(x), x)$ and $\neg \text{Loves}(u, v)$, with unifier $\theta = \{u/G(x), v/x\}$, to produce the resolvent clause

$[\text{Animal}(F(x)) \vee \neg \text{Kills}(G(x), x)]$

11*Resolution Refutation (Proof by contradiction):

Resolution refutation proves that $KB \models \alpha$ by proving $KB \wedge \neg \alpha$ unsatisfiable, that is, by deriving the empty clause.

Main resolution refutation steps:

1.Convert KB and $\neg \alpha$ to CNF with ground terms and universal variable only.

2.Apply repeatedly the resolution rule.

3.Stop when empty set (contradiction) is derived or no more resolvents (conclusions) follow.

If we derive a contradiction, then the conclusion follow from the axioms.

If we can't apply anymore resolvents, then the conclusion cannot be proved from the axioms.

Resolution Refutation is sound and complete.

Example:

The sentences in CNF are:

$\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$

$\neg \text{Missile}(x) \vee \neg \text{Owns}(\text{Nono}, x) \vee \text{Sells}(\text{West}, x, \text{Nono})$

$\neg \text{Enemy}(x, \text{America}) \vee \text{Hostile}(x)$

$\neg \text{Missile}(x) \vee \text{Weapon}(x)$

$\text{Owns}(\text{Nono}, M1)$

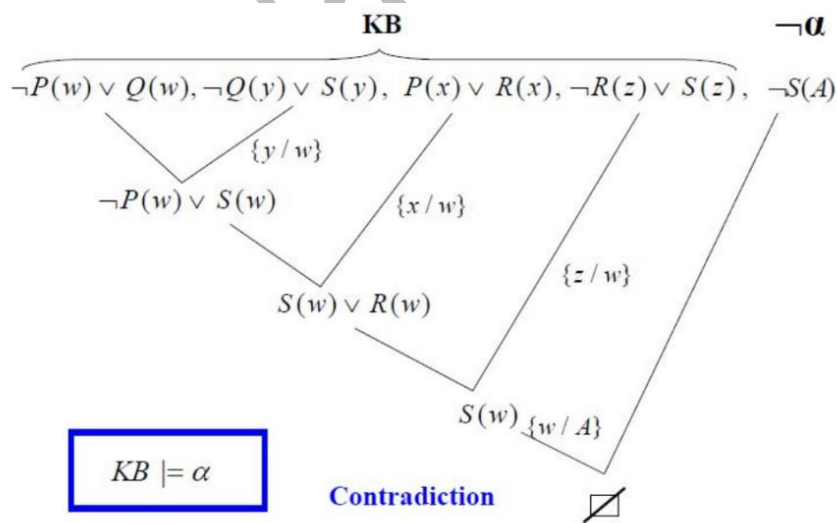
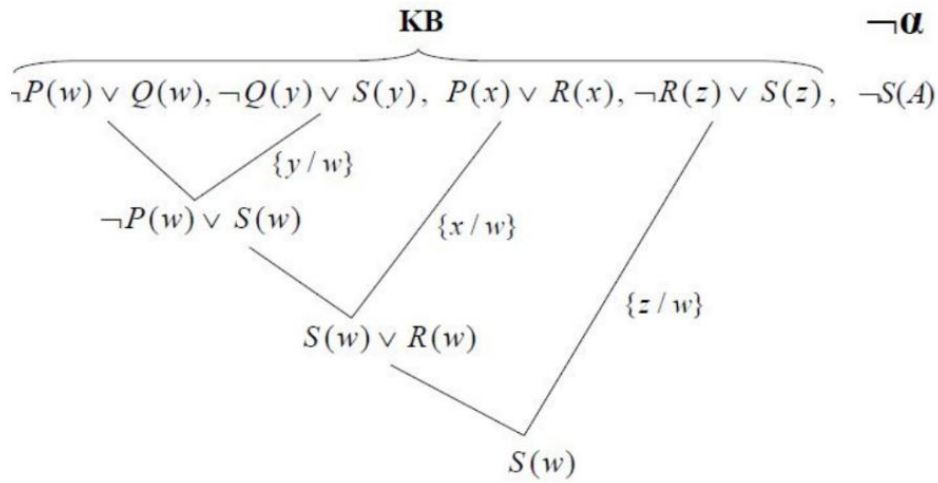
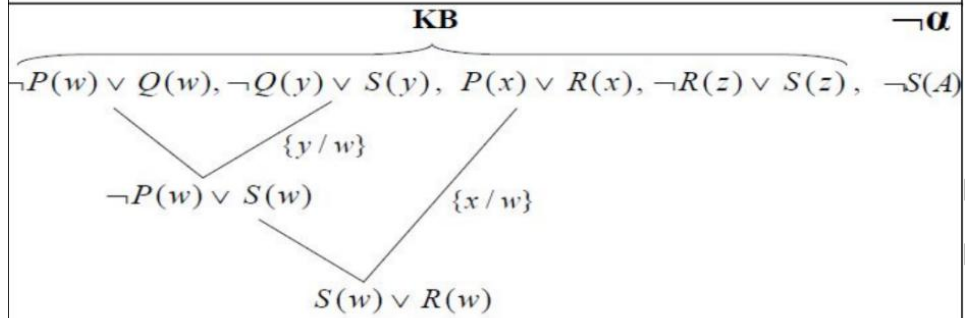
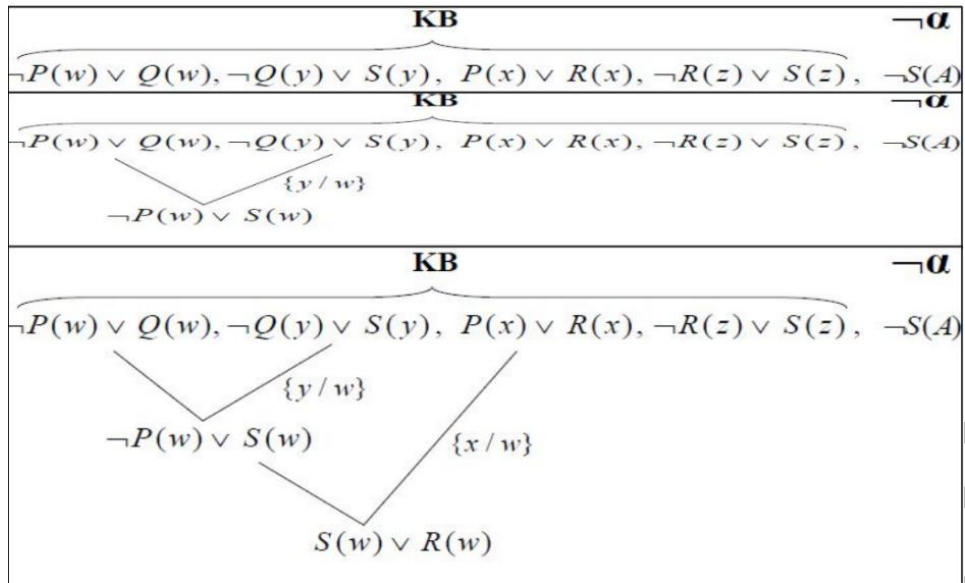
$\text{Missile}(M1)$

$\text{American}(\text{West})$

$\text{Enemy}(\text{Nono}, \text{America})$.

We also include the negated goal

$\neg \text{Criminal}(\text{West})$.



Discrete Mathematics:

Chapter 01: The Foundations – Logic and Proofs[1]

01*Positional (Propositional) Logic:

What is a Proposition?

A proposition is a statement that has a definite truth value. It is either true (T) or false (F), but not both.

Examples:

- 7 is greater than 5. $\rightarrow$ True

- Paris is the capital of Italy. $\rightarrow$ False

Questions, commands, and open sentences (such as $x > 3$) are not propositions because they do not have a fixed truth value.

Compound Propositions:

A compound proposition is formed by combining one or more propositions using logical operators.

1.Negation ($\neg p$):

Negation changes the truth value of a proposition.

- If p is true, then $\neg p$ is false.

- If p is false, then $\neg p$ is true.

Example:

- p : "It is raining."

- $\neg p$: "It is not raining."

2.Conjunction ($p \wedge q$):

A conjunction means " p and q ."

It is true only when both propositions are true.

Example:

- p : "I finished my homework."

- q : "I submitted the assignment."

- $p \wedge q$: "I finished my homework and submitted the assignment."

3.Disjunction ($p \vee q$):

A disjunction means " p or q ."

It is false only when both propositions are false.

Example:

- p : "I will study."

- q : "I will exercise."

- $p \vee q$ means at least one of them is true.

4. Conditional Statement ($p \rightarrow q$):

A conditional statement means "If p, then q."

It is false only when p is true and q is false.

Example:

- If it rains, then the ground becomes wet.

5. Biconditional Statement ($p \leftrightarrow q$):

A biconditional means "p if and only if q."

It is true when both propositions have the same truth value (both true or both false).

Example:

- A figure is a square if and only if it has four equal sides and four right angles.

Truth Table:

A truth table shows the truth value of a compound proposition for every possible combination of truth values.

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Truth tables are useful for checking logical relationships and verifying arguments.

Converse, Inverse, and Contrapositive:

Given the conditional statement:

$p \rightarrow q$ (If p, then q)

Three related statements can be formed.

1. Converse:

Swap the hypothesis and conclusion.

$q \rightarrow p$

Example:

- Original: If it rains, then the road is wet.

- Converse: If the road is wet, then it is raining.

The converse is not logically equivalent to the original statement.

2. Inverse:

Negate both parts of the original statement.

$\neg p \rightarrow \neg q$

Example:

- If it does not rain, then the road is not wet.

The inverse is also not logically equivalent to the original statement.

3. Contrapositive:

Swap the hypothesis and conclusion, then negate both.

$$\neg q \rightarrow \neg p$$

Example:

- If the road is not wet, then it did not rain.

The contrapositive is logically equivalent to the original conditional statement. This means they always have the same truth value.

Chapter 02 : The Foundations – Logic and Proofs[2]:

01*Propositional Equivalence:

Tautology, Contradiction, and Contingency

Logical statements can be classified based on their truth values.

1. Tautology:

A tautology is a proposition that is always true, no matter what truth values are assigned to its variables.

Example:

$$p \vee \neg p$$

This expression is always true because either p is true or p is false.

2. Contradiction:

A contradiction is a proposition that is always false, regardless of the truth values of its variables.

Example:

$$p \wedge \neg p$$

A proposition cannot be both true and false at the same time, so this expression is always false.

3. Contingency:

A contingency is a proposition that is true in some cases and false in others. Its truth value depends on the values of its variables.

Example:

$$p \rightarrow q$$

Since its truth value changes with different inputs, it is neither a tautology nor a contradiction.

Logical Equivalence:

Two propositions are logically equivalent if they always have the same truth value for every possible combination of variables.

The symbol for logical equivalence is $\equiv$.

Example:

$$\neg \neg p \equiv p$$

To verify logical equivalence, construct a truth table for both propositions. If the final columns are identical, the propositions are logically equivalent.

Common Logical Equivalence Laws:

Law	Equivalent Expression
Double Negation	$p \equiv \neg \neg p$
Identity Laws	$p \wedge T \equiv p$, $p \vee F \equiv p$
Domination Laws	$p \vee T \equiv T$, $p \wedge F \equiv F$
Idempotent Laws	$p \vee p \equiv p$, $p \wedge p \equiv p$
Negation Laws	$p \vee \neg p \equiv T$, $p \wedge \neg p \equiv F$
Commutative Laws	$p \vee q \equiv q \vee p$, $p \wedge q \equiv q \wedge p$
Associative Laws	$(p \vee q) \vee r \equiv p \vee (q \vee r)$, $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
Distributive Laws	$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$, $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
De Morgan's Laws	$\neg(p \wedge q) \equiv \neg p \vee \neg q$, $\neg(p \vee q) \equiv \neg p \wedge \neg q$

02*Predicates and Quantifiers:

Predicate:

A predicate is a statement that contains one or more variables. Unlike a proposition, a predicate does not have a truth value until the variables are assigned specific values.

A predicate is usually written as $P(x)$, where x is the variable.

Examples:

- " $P(x): x > 5$ "

- " $Q(x): x$ is an even number"

These statements become propositions only after replacing x with a specific value.

Example:

- " $P(8): 8 > 5 \rightarrow \text{True}$ "

- " $P(3): 3 > 5 \rightarrow \text{False}$ "

Quantifiers:

A quantifier specifies how many elements in a given set satisfy a predicate. The two main quantifiers are the universal quantifier and the existential quantifier.

1.Universal Quantifier (For All):

The universal quantifier is written as $\forall x$ and means "for every element in the domain."

Notation:

- " $\forall x P(x)$ "

This statement is true only if $P(x)$ is true for every value of x in the domain.

Example:

- " $\forall x (x + 0 = x)$ "

This statement is true because adding zero does not change any number.

2.Existential Quantifier (There Exists):

The existential quantifier is written as $\exists x$ and means "there is at least one element in the domain."

Notation:

- " $\exists x P(x)$ "

This statement is true if at least one value of x satisfies the predicate.

Example:

- " $\exists x (x^2 = 16)$ "

This statement is true because 4 and -4 satisfy the equation.

Logical Equivalences Involving Quantifiers:

Negating a quantified statement changes both the quantifier and the predicate.

Negation of the Universal Quantifier

" $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$ "

Meaning:

- "It is not true that every element satisfies P" is equivalent to "There exists at least one element that does not satisfy P."

Negation of the Existential Quantifier:

" $\neg(\exists x P(x)) \equiv \forall x \neg P(x)$ "

Meaning:

- "There does not exist an element that satisfies P" is equivalent to "Every element does not satisfy P."

These equivalence rules are important when simplifying logical expressions and proving mathematical statements.

03*Nested Quantifiers:

Nested quantifiers occur when a statement contains more than one quantifier. The order of the quantifiers is important because changing the order may change the meaning of the statement.

Examples:

- " $\forall x \exists y P(x, y)$ "

- " $\exists x \forall y P(x, y)$ "

These two statements are generally not equivalent.

Example:

" $\forall x \exists y (x + y = 0)$ "

Meaning:

For every value of x, there is a value of y such that their sum is zero.

This statement is true because for any number x, we can choose $y = -x$.

Another example:

" $\exists x \forall y (x + y = y)$ "

Meaning: There exists a number x that, when added to any number y, does not change its value.

This statement is true because $x = 0$ satisfies the condition.

04*Mathematical Statements Involving Quantifiers:

Quantifiers are commonly used to write mathematical definitions and theorems in a precise way.

Examples:

- " $\forall x (x + 0 = x)$ "

Meaning: Every number remains the same when zero is added.

- " $\exists x (x^2 = 25)$ "

Meaning: There is at least one number whose square is 25.

- " $\forall x (x^2 \geq 0)$ "

Meaning: The square of every real number is non-negative.

These statements clearly describe mathematical properties using quantifiers.

05*Mathematical Statements Involving Nested Quantifiers:

Nested quantifiers are often used to describe relationships between two or more variables.

Example 1:

" $\forall x \exists y (y > x)$ "

Meaning: For every number x , there exists another number y that is greater than x .

This statement is true for the set of real numbers.

Example 2:

" $\exists x \forall y (x \geq y)$ "

Meaning: There exists one number that is greater than or equal to every number in the domain.

This statement is false for the set of real numbers because there is no largest real number.

Note:

The order of quantifiers matters.

For example:

- " $\forall x \exists y P(x, y)$ " means each value of x may have a different value of y .

- " $\exists y \forall x P(x, y)$ " means one fixed value of y must work for every value of x .

Although the statements use the same quantifiers, they usually have different meanings and truth values.

Chapter 01: Introduction to Machine Learning

1.Compiler / Interpreter (Definitions):

Compiler

A compiler translates the entire source code into machine code before the program runs.

The program is executed only after compilation is complete.

Compiled programs usually run faster.

Interpreter

An interpreter translates and executes the source code one statement at a time.

Errors are reported immediately when they are found.

It is useful for testing and debugging.

2.How an Interpreter Works (Schema):

Source Code $\rightarrow$ Interpreter $\rightarrow$ Execute Line by Line $\rightarrow$ Output

Steps:

The programmer writes the source code.

The interpreter reads one line of code.

It translates the line into machine instructions.

The line is executed immediately.

The process continues until the program finishes or an error occurs.

3.Data Science, Machine Learning, and Artificial Intelligence (Definitions):

Data Science

A field that collects, analyzes, and interprets data to discover useful information and support decision-making.

Artificial Intelligence (AI)

A branch of computer science that enables machines to perform tasks that normally require human intelligence, such as learning, reasoning, and problem-solving.

Machine Learning (ML)

A branch of AI that allows computers to learn from data and improve their performance without being explicitly programmed.

4.What is Machine Learning?

Machine Learning is the process of training computers to recognize patterns in data and make predictions or decisions automatically.

Instead of following fixed rules, ML systems improve by learning from experience (data).

5.Why Use Machine Learning?

To analyze large amounts of data quickly.

To make accurate predictions.

To automate repetitive tasks.

To detect hidden patterns and relationships in data.

To improve decision-making.

To continuously improve performance as more data becomes available.

Keywords to Remember: Compiler, Interpreter, Data Science, Artificial Intelligence (AI), Machine Learning (ML), Data, Prediction, Pattern Recognition, Automation.

6.Applications of Machine Learning:

Machine Learning is used in many real-world applications, including:

Image and face recognition.

Speech recognition and voice assistants.

Email spam detection.

Recommendation systems (movies, music, online shopping).

Medical diagnosis and healthcare.

Fraud detection in banking.

Self-driving cars.

Predictive maintenance in industry.

2.Terminology:

Dataset

A collection of data used to train and evaluate a machine learning model.

Features

The input variables or characteristics used to make predictions.

Training Data

The portion of the dataset used to teach the model by finding patterns.

Testing Data

The portion of the dataset used to evaluate how well the trained model performs on new data.

Model

The final learning system created from the training data that can make predictions or decisions.

3.Traditional Programming:

In traditional programming:

Input Data + Program (Rules) → Output

The programmer writes all the rules.

The computer follows these predefined rules to produce the output.

4.How Does Machine Learning Work? (Basic Learning Process):

Data Input

The model receives examples from a dataset.

Abstraction

The model learns patterns and relationships from the data.

Generalization

The model applies the learned patterns to make predictions on new, unseen data.

5.Machine Learning Process:

- Collect data
- Prepare and clean the data.
- Select useful features.
- Train the machine learning model.
- Test and evaluate the model.
- Improve the model if necessary.
- Use the trained model to make predictions.

6.Machine Learning Types:

Supervised Learning:

The model learns from labeled data (input and correct output are provided).

Unsupervised Learning:

The model learns from unlabeled data and discovers hidden patterns or groups.

Reinforcement Learning

The model learns through trial and error by receiving rewards or penalties for its actions.

Keywords to Remember:

Dataset

Features

Training Data

Testing Data

Model

Traditional Programming

Data Input

Abstraction

Generalization

Supervised Learning

Unsupervised Learning

Reinforcement Learning

Dataset:

A dataset is a collection of data used to train and evaluate a machine learning model.

Features:

Features are the input variables or characteristics used by a model to make predictions.

Training Data:

Training data is the data used to teach a machine learning model by learning patterns.

Testing Data:

Testing data is the data used to evaluate the performance of a trained model.

Model:

A model is a trained system that makes predictions or decisions based on data.

Traditional Programming

Traditional programming uses predefined rules written by a programmer to produce the correct output.

Data Input:

Data input is the information provided to a machine learning model for learning or prediction.

Abstraction:

Abstraction is the process of learning patterns and relationships from the input data.

Generalization:

Generalization is the ability of a model to make accurate predictions on new, unseen data.

Supervised Learning:

Supervised learning is a type of machine learning that uses labeled data for training.

Unsupervised Learning:

Unsupervised learning is a type of machine learning that finds patterns in unlabeled data.

Reinforcement Learning:

Reinforcement learning is a type of machine learning where an agent learns through rewards and penalties.

7. Machine Learning Algorithms:

Machine learning algorithms are mathematical methods that enable computers to learn patterns from data and make predictions or decisions.

Examples:

Linear Regression

Decision Tree

K-Nearest Neighbors (KNN)

Support Vector Machine (SVM)

Naïve Bayes

Neural Networks

8. Machine Learning Process:

The basic machine learning process consists of:

Collect data.

Prepare and clean the data.

Train the model.

Test and evaluate the model.

Deploy the model to make predictions.

9. Machine Learning Software Tools:

Common software tools used in machine learning include:

Python : The most popular programming language for ML.

Jupyter Notebook : An interactive environment for writing and running code.

Scikit-learn : A library that provides many machine learning algorithms.

TensorFlow : A framework for building machine learning and deep learning models.

Keras : A high-level API for creating neural networks.

Pandas : Used for data manipulation and analysis.

NumPy : Used for numerical and mathematical operations.

Matplotlib : Used for data visualization and plotting graphs.

10. Machine Learning Libraries: NumPy

1. NumPy:

Definition: NumPy (Numerical Python) is a Python library used for numerical computing and working efficiently with arrays and mathematical operations.

Example:

```
import numpy as np
```

2. range():

Definition: The range() function generates a sequence of integers. It is commonly used in loops.

Example Code:

```
for i in range(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

3. np.arange():

Definition: np.arange() creates a NumPy array with evenly spaced values.

One Parameter

```
import numpy as np  
x = np.arange(5)  
print(x)
```

Output:

```
[0 1 2 3 4]
```

Two Parameters

```
x = np.arange(2, 7)
print(x)
```

Output:

```
[2 3 4 5 6]
```

Three Parameters

```
x = np.arange(0, 10, 2)
print(x)
```

Output:

```
[0 2 4 6 8]
```

4. NumPy Array:

Definition: A NumPy array is a data structure that stores multiple values of the same type efficiently.

Example:

```
import numpy as np
arr = np.array([10, 20, 30, 40])
print(arr)
```

Output:

```
[10 20 30 40]
```

5. Array & Shape:

Definition: The shape of an array shows the number of rows and columns (dimensions) in the array.

Example:

```
import numpy as np
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
print(arr.shape)
```

Output:

```
(2, 3)
(2 rows, 3 columns)
```

6. np.ones():

Definition: np.ones() creates an array filled with the value 1.

Example:

```
import numpy as np
arr = np.ones((2, 3))
print(arr)
```

Output:

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

7.np.random:

Definition: np.random is used to generate random numbers or random arrays.

Example:

```
import numpy as np
arr = np.random.rand(2, 2)
print(arr)
```

Example Output:

```
[[0.45 0.81]
 [0.12 0.67]]
(The values are different each time the program runs.)
```

11.Machine Learning Libraries: Pandas

1.Pandas (Series / DataFrame):

Definition: Pandas is a Python library used for data manipulation and analysis. Its two main data structures are Series and DataFrame.

Example:

```
import pandas as pd
```

2.Pandas Series:

Definition: A Pandas Series is a one-dimensional labeled array that can store different types of data.

Example:

```
import pandas as pd
s = pd.Series([10, 20, 30, 40])
print(s)
```

Output:

```
0    10
1    20
2    30
3    40
dtype: int64
```

3.pd.Series():

Definition: pd.Series() is a constructor used to create a Pandas Series from a list, tuple, array, or dictionary.

Example:

```
import pandas as pd
data = pd.Series([5, 10, 15])
print(data)
```

Output:

```
0     5
1    10
2    15
dtype: int64
```

4. Pandas Series Index:

Definition: The index is the label assigned to each element in a Series. By default, indexing starts from 0, but custom indexes can also be used.

Example:

```
import pandas as pd
s = pd.Series([100, 200, 300], index=["A", "B", "C"])
print(s)
```

Output:

```
A    100
B    200
C    300
dtype: int64
```

5. Pandas with Dictionary:

Definition: A dictionary can be used to create a Pandas Series or DataFrame, where dictionary keys become labels or column names.

Example:

```
import pandas as pd
data = {"Ali": 90, "Sara": 95, "Omar": 88}
s = pd.Series(data)
print(s)
```

Output:

```
Ali    90
Sara   95
Omar   88
dtype: int64
```

6. Pandas with Dataset (DataFrame):

Definition: A DataFrame is a two-dimensional table consisting of rows and columns. It is commonly used to store and analyze datasets.

Example:

```
import pandas as pd
data = {
    "Name": ["Ali", "Sara", "Omar"],
    "Age": [20, 21, 22]
}
df = pd.DataFrame(data)
print(df)
```

Output:

```
   Name  Age
0  Ali   20
1  Sara  21
2  Omar  22
```

Pandas: Data analysis library.

Series: One-dimensional labeled array.

DataFrame: Two-dimensional table of rows and columns.

Index: Labels that identify Series elements.

Dictionary: Key-value data structure used to create Series or DataFrames.

Dataset: A collection of organized data stored in a DataFrame.

12.DataFrame Methods:

1.df.head():

Definition: Returns the first 5 rows of the DataFrame by default.

Example:

```
import pandas as pd
df = pd.read_csv("data.csv")
print(df.head())
```

Output:

```
Name Age
0 Ali 20
1 Sara 21
2 Omar 22
3 Noor 19
4 Rami 23
```

2.df.tail():

Definition: Returns the last 5 rows of the DataFrame by default.

Example:

```
print(df.tail())
```

Output:

```
Name Age
5 Lina 20
6 Ahmad 24
7 Maya 21
8 Zaid 22
9 Reem 20
```

3.df.count():

Definition: Returns the number of non-missing values in each column.

Example:

```
print(df.count())
```

Output:

```
Name 10
Age 10
dtype: int64
```

4.df.info() :

Definition: Displays information about the DataFrame.

Example:

```
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 2 columns):
Name 10 non-null object
Age 10 non-null int64
dtypes: int64(1), object(1)
```

5.df.describe()

Definition: Generates statistical summaries for numerical columns.

Example:

```
print(df.describe())
```

Output:

```
Age
count 10.000000
mean  21.200000
std    1.55
min   19.000000
max   24.000000
```

6.df.shape

Definition: Returns the number of rows and columns.

Example:

```
print(df.shape)
```

Output:

```
(10, 2)
```

7.df.columns

Definition: Returns the names of all columns.

Example:

```
print(df.columns)
```

Output:

```
Index(['Name', 'Age'], dtype='object')
```

13. Matplotlib:

1. Matplotlib:

Definition: Matplotlib is a Python library used to create charts and graphs for data visualization.

Example:

```
import matplotlib.pyplot as plt
```

2. matplotlib.pyplot:

Definition: matplotlib.pyplot is a module used to create and customize graphs.

Example:

```
import matplotlib.pyplot as plt
```

3. plt.figure()

Definition: Creates a new figure (plot window).

Example:

```
import matplotlib.pyplot as plt
plt.figure()
plt.show()
```

Output:

```
A blank figure window appears.
```

4.plt.scatter()

Definition: Creates a scatter plot to show the relationship between two variables.

Example:

```
import matplotlib.pyplot as plt
x = [1, 2, 3]
y = [2, 4, 6]
plt.scatter(x, y)
plt.show()
```

Output:

```
(1,2), (2,4), (3,6)
```

5.plt.bar()

Definition: Creates a bar chart to compare values.

Example:

```
import matplotlib.pyplot as plt
names = ["A", "B", "C"]
marks = [80, 90, 70]
plt.bar(names, marks)
plt.show()
```

Output:

```
A = 80
B = 90
C = 70
```

6.plt.show()

Definition: Displays the graph.

Example:

```
plt.show()
```

Output:

```
The graph is displayed.
```

14.Simple Plot:

Definition: A simple plot is a basic line graph used to visualize data.

Example:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]
plt.plot(x, y)
plt.show()
```

Output:

```
A line graph connecting the points:
(1,2), (2,4), (3,6), (4,8)
```

15. Plot with NumPy:

Definition: NumPy arrays can be used directly to create plots.

Example:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([1, 2, 3, 4])
y = np.array([1, 4, 9, 16])
plt.plot(x, y)
plt.show()
```

Output:

A line graph connecting:
(1,1), (2,4), (3,9), (4,16)

16. Pyplot Marker:

Definition: A marker highlights each data point on a graph.

Example:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]
plt.plot(x, y, marker='o')
plt.show()
```

Output:

A line graph with circle (o) markers at each point.

17. Plotting Data:

Definition: Plotting data means representing data visually using graphs.

Example:

```
import matplotlib.pyplot as plt
x = [10, 20, 30]
y = [15, 25, 35]
plt.plot(x, y)
plt.show()
```

Output:

A line graph connecting:
(10,15), (20,25), (30,35)

18. Pyplot (plt):

Definition: Pyplot (plt) is the commonly used alias for the matplotlib.pyplot module, providing functions to create and customize graphs.

Example:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [2, 4, 6])
plt.title("Simple Plot")
plt.show()
```

Output:

A line graph titled:
Simple Plot

Chapter 02: Data Exploration

01*Data Exploration:

Definition: Data Exploration is the process of examining and understanding a dataset before analysis or machine learning.

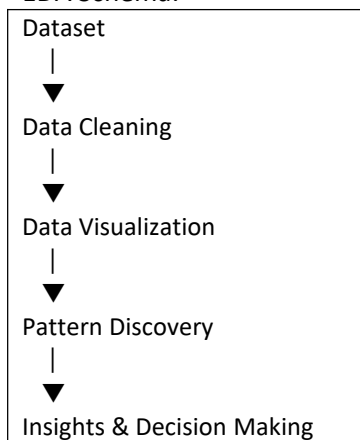
It allows us to:

- Understand the structure of the data.
- Identify patterns and relationships.
- Detect missing values and errors.
- Find outliers.
- Prepare data for analysis and modeling.

02*EDA (Exploratory Data Analysis):

Definition: Exploratory Data Analysis (EDA) is the process of summarizing and visualizing data to better understand its characteristics before building a machine learning model.

EDA Schema:



03*Dataset:

Definition: A dataset is a collection of organized data stored in rows and columns.

A.Qualitative Data (Categorical Data):

Definition: Qualitative data describes characteristics or categories rather than numerical values.

Categories

Data grouped into different classes.

Example: Male, Female.

Attribute: Descriptive properties of an object.

Example: Color, Brand, City.

B.Quantitative Data (Numerical Data):

Definition: Quantitative data consists of numerical values that can be measured or counted.

Discrete Data : Countable numerical values.

Example: Number of students, Number of books.

Continuous Data : Measurable values that can take any value within a range.

Example: Height, Weight, Temperature.

04*Example of a Dataset (Table):

Student	Gender	Age	Height (cm)	Grade
Ali	Male	20	175	88
Sara	Female	21	165	92
Omar	Male	22	180	85
Noor	Female	20	168	90

Column Types:

Student : Qualitative (Attribute)

Gender : Qualitative (Category)

Age : Quantitative (Discrete)

Height : Quantitative (Continuous)

Grade : Quantitative (Discrete)

05*Kaggle:

Definition: Kaggle is an online platform for data science and machine learning. It provides datasets, notebooks, competitions, and learning resources.

Some Features of Kaggle:

- Large collection of free datasets.
- Machine learning competitions.
- Online coding environment (Kaggle Notebooks).
- Courses for learning data science and ML.
- Community for sharing projects and solutions.

06*Data Exploration in Python Using Pandas:

A.Creating a DataFrame from an Excel File

Definition: The `read_excel()` function reads data from an Excel file and stores it in a DataFrame.

Example:

```
import pandas as pd
df = pd.read_excel("students.xlsx")
print(df)
```

Output:

	Name	Age
0	Ali	20
1	Sara	21
2	Omar	22

B.Creating a DataFrame from a CSV File:

Definition: The `read_csv()` function reads data from a CSV file and stores it in a DataFrame.

Example:

```
import pandas as pd
df = pd.read_csv("students.csv")
print(df)
```

Output:

	Name	Age
0	Ali	20
1	Sara	21
2	Omar	22

07*Viewing Data:

Definition: Viewing data means displaying part or all of a DataFrame to understand its contents.

Example:

```
print(df.head())
```

Output:

	Name	Age
0	Ali	20
1	Sara	21
2	Omar	22

08*Searching Data:

Definition: Searching data means selecting rows that satisfy a specific condition.

Example:

```
print(df[df["Age"] > 20])
```

Output:

	Name	Age
1	Sara	21
2	Omar	22

09*Searching and Removing Data:

Definition: Rows can be searched using a condition and removed using the drop() function.

Example:

```
df = df.drop(df[df["Age"] == 20].index)
print(df)
```

Output:

	Name	Age
1	Sara	21
2	Omar	22

10*Updating Data:

Definition: Updating data means changing the value of one or more cells in a DataFrame.

Example:

```
df.loc[0, "Age"] = 25
print(df)
```

Output:

	Name	Age
0	Ali	25
1	Sara	21
2	Omar	22

11*Some Keywords Definitions till now :

Kaggle: Data science platform.

read_excel() : Read an Excel file.

read_csv() : Read a CSV file.

Viewing Data : Display dataset.

Searching Data : Filter rows.

Removing Data : Delete rows.

Updating Data : Modify values in a DataFrame.

12*Finding Relationships:

Definition: Finding relationships means identifying connections or correlations between variables in a dataset.

Example:

```
import pandas as pd
df = pd.read_csv("students.csv")
print(df.corr(numeric_only=True))
```

Output:

	Age	Grade
Age	1.00	0.72
Grade	0.72	1.00

13*Data Scrubbing (Data Cleaning):

Definition: Data Scrubbing (Data Cleaning) is the process of detecting and correcting inaccurate, incomplete, or inconsistent data before analysis.

Example:

```
import pandas as pd
df = pd.read_csv("students.csv")
df = df.dropna()
print(df)
```

Output:

	Name	Age	Grade
0	Ali	20	88
2	Omar	22	85

14*Missing Data:

Definition: Missing data refers to values that are not available or are absent from the dataset.

Example:

```
print(df.isnull())
```

Output:

	Name	Age	Grade
0	False	False	False
1	False	True	False
2	False	False	False

15*Null Values:

Definition: Null values are empty or undefined values in a dataset and are usually represented as NaN.

Example:

```
print(df.isnull().sum())
```

Output:

Name	0
Age	1
Grade	0

dtype: int64

16*Wrong Format:

Definition: Wrong format means data is stored in an incorrect type or format, such as dates stored as text.

Example:

```
df["Date"] = pd.to_datetime(df["Date"])
print(df)
```

Output:

```
   Date
0 2026-01-10
1 2026-01-15
2 2026-01-20
```

17*Duplicate Values:

Definition: Duplicate values are repeated rows in a dataset that should be identified and removed.

Example:

```
df = df.drop_duplicates()
print(df)
```

Output:

Duplicate rows removed.

18*Pandas Plotting:

Definition: Pandas plotting is used to create charts directly from a DataFrame using the plot() function.

Example:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({
    "Year": [2022, 2023, 2024],
    "Sales": [100, 150, 200]
})
df.plot(x="Year", y="Sales")
plt.show()
```

Output:

A line graph showing Sales increasing from:
2022 → 100
2023 → 150
2024 → 200

Some Keywords:

Finding Relationships: Identify connections between variables.

Data Scrubbing (Cleaning): Correct and prepare data.

Missing Data: Data that is not available.

Null Values: Empty values (NaN).

Wrong Format: Incorrect data type or format.

Duplicate Values: Repeated rows.

Pandas Plotting: Create charts using the plot() function.

Chapter 03: Types of Machine Learning & First Machine Learning

01*Types of Machine Learning:

A.Supervised Learning:

Definition: Supervised learning is a type of machine learning in which the model is trained using labeled data, where the correct output is already known.

How it works:

Input data + Correct output (label).

The model learns the relationship between inputs and outputs.

It predicts the output for new data.

Example: Predicting whether an email is Spam or Not Spam.

B.Unsupervised Learning:

Definition: Unsupervised learning is a type of machine learning in which the model learns from unlabeled data and discovers hidden patterns or groups.

How it works:

Input data only (no labels).

The model groups similar data together.

Example: Grouping customers based on their shopping behavior.

C.Reinforcement Learning:

Definition: Reinforcement learning is a type of machine learning in which an agent learns by interacting with an environment and receiving rewards or penalties.

How it works:

The agent performs an action.

The environment gives feedback.

The agent improves its future actions.

Example: A robot learning to walk or a game-playing AI learning to win.

02*Types of Problems in Machine Learning

A.Regression:

Definition: Regression predicts a continuous numerical value.

Example: Predicting house prices or future temperatures.

B.Classification:

Definition: Classification predicts a category or class.

Example: Spam or Not Spam, Pass or Fail, Healthy or Sick.

C.Clustering:

Definition: Clustering groups similar data together without using labels.

Example: Grouping customers with similar buying habits.

03*Machine Learning Algorithms:

Definition: Machine learning algorithms are mathematical methods that enable computers to learn patterns from data and make predictions or decisions.

04*Machine Learning Algorithms and Their Use Cases:

A.Linear Regression:

Definition: A supervised learning algorithm used to predict continuous numerical values.

Use Case:

- House price prediction.
- Sales forecasting.
- Temperature prediction.

B.Logistic Regression:

Definition: A supervised learning algorithm used for classification problems.

Use Case:

- Spam email detection.
- Disease prediction.
- Customer churn prediction.

C.Decision Tree:

Definition: A supervised learning algorithm that makes decisions using a tree-like structure.

Use Case:

- Loan approval.
- Medical diagnosis.
- Student performance prediction.

D.Naïve Bayes:

Definition: A supervised learning algorithm based on probability and Bayes' Theorem.

Use Case:

- Spam filtering.
- Sentiment analysis.
- Text classification.

E.Support Vector Machine (SVM):

Definition: A supervised learning algorithm used for classification and regression by finding the best boundary between classes.

Use Case:

- Face recognition.
- Image classification.
- Handwriting recognition.

F.Random Forest:

Definition: A supervised learning algorithm that combines multiple decision trees to improve prediction accuracy.

Use Case:

- Fraud detection.
- Disease diagnosis.
- Credit risk analysis.

G.K-Means Clustering:

Definition: An unsupervised learning algorithm that groups similar data into K clusters.

Use Case:

- Customer segmentation.
- Image compression.
- Market analysis.

To Remember:

Supervised Learning : Labeled data.
Unsupervised Learning : Unlabeled data.
Reinforcement Learning : Rewards and penalties.
Regression : Predict numerical values.
Classification : Predict categories.
Clustering : Group similar data.
Linear Regression : Continuous prediction.
Logistic Regression : Classification.
Decision Tree : Tree-based decisions.
Naïve Bayes : Probability-based classification.
Support Vector Machine (SVM) : Best separating boundary.
Random Forest : Multiple decision trees.
K-Means : Clustering algorithm.

05*Machine Learning Process (Progress):**1.Find the Data:**

Finding data means collecting relevant information from files, databases, or online sources to use in machine learning.

2.Clean the Data:

Cleaning data means removing errors, missing values, duplicates, and fixing wrong formats to make the dataset usable.

3.Split the Data into Training:

Splitting data means dividing the dataset into training data (to learn) and testing data (to evaluate performance).

4.Create a Model:

Creating a model means selecting a machine learning algorithm that will learn patterns from the data.

5.Train the Model:

Training the model means feeding it training data so it can learn relationships between inputs and outputs.

6.Make Prediction:

Making prediction means using the trained model to predict outputs for new, unseen data.

7.Evaluate and Improve:

Evaluating and improving means checking model accuracy and adjusting it to perform better if needed.

06*Import the Data:

Definition: Importing data means loading datasets into Python using libraries like Pandas.

Example:

```
import pandas as pd
df = pd.read_csv("data.csv")
print(df.head())
```


07*Intro to Scikit-learn (Sklearn):

Definition: Scikit-learn (sklearn) is a Python library used to build and train machine learning models easily.

It provides:

- Algorithms (classification, regression, clustering)
- Data preprocessing tools
- Model evaluation tools

Import Example:

```
import sklearn
```

08*Create a Model:

Definition: Creating a model means choosing a machine learning algorithm from sklearn to build a predictive system.

Example:

```
from sklearn.tree import DecisionTreeClassifier  
model = DecisionTreeClassifier()
```

09*Train a Model:

Definition: Training a model means fitting it to the training data so it learns patterns between inputs and outputs.

Example:

```
model.fit(X_train, y_train)
```

X_train : input features (data)

y_train : correct labels (answers)

10*Machine Learning: Prediction with predict()

1.Dataset: music.csv:

Definition: A dataset used to train a model that predicts music preference based on user features (example: age, gender, etc.).

Example Structure:

Age	Gender	Music
20	1	Rock
25	0	Jazz
30	1	Pop

2.Split Data into Training / Test Sets:

Definition: Splitting data means dividing it into:

Training set: used to train the model

Test set: used to evaluate the model

Example:

```
import pandas as pd  
from sklearn.model_selection import train_test_split  
df = pd.read_csv("music.csv")  
X = df.drop(columns=["genre"]) # input features  
y = df["genre"] # output labels  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

3. Machine Learning Algorithm:

We use a Decision Tree Classifier to learn patterns.

Definition: A Decision Tree splits data into branches based on conditions to make predictions.

Example:

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
```

4. Train the Model:

Definition: Training means the model learns relationships from training data.

Example:

```
model.fit(X_train, y_train)
```

5. Make Predictions (predict()):

Definition:

predict() is used to make predictions on new or test data using a trained model.

Example:

```
predictions = model.predict(X_test)
print(predictions)
```

Output:

```
['Rock', 'Jazz', 'Pop', 'Rock']
```

6. Full Workflow Summary:

- Machine Learning Process
- Load dataset (music.csv)
- Split into training & testing sets
- Choose algorithm (Decision Tree)
- Train model using fit()
- Make predictions using predict()

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
# Load data
df = pd.read_csv("music.csv")
# Split data
X = df.drop(columns=["genre"])
y = df["genre"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
# Create model
model = DecisionTreeClassifier()
# Train model
model.fit(X_train, y_train)
# Make predictions
predictions = model.predict(X_test)
print(predictions)
```

11*Machine Learning Terminology:

1.Features / Attributes:

Definition: Features (or attributes) are the input variables used by a machine learning model to make predictions.

Example: Age, Gender, Height, Income.

2.Label / Target:

Definition: The label (or target) is the correct output that the model is trying to predict.

Example: Music genre (Rock, Jazz), Price, Yes/No.

3.Training Data:

Definition: Training data is the part of the dataset used to teach the model how inputs relate to outputs.

4.Observations:

Definition: An observation is a single data point in a dataset (one row of data).

5.Sample:

Definition: A sample is another name for one complete entry in a dataset, containing all features and a label.

6.Dataset (Samples):

Definition: A dataset is a collection of many samples (many rows of data).

7.Row:

Definition: A row represents one observation/sample in a dataset.

8.Testing Data:

Definition: Testing data is the part of the dataset used to evaluate how well the trained model performs on new data.

9.Model:

Definition: A model is a trained system that learns patterns from data and uses them to make predictions.

Chapter 04 : Linear Regression algorithm

01*Classification, Regression:

- Regression Predict a real number associated with a feature vector
– E.g., use linear regression to fit a curve to data
- Classification Predict a discrete value (label) associated with a feature vector

The regression technique finds out a linear relationship between input and output, hence, the name linear regression.

02*What is a Linear regression algorithm?

•Regression is a very simple approach to supervised learning statistical method to establish a relationship between a dependent variable and a set of independent variables

Examples of a Linear regression:

Relation between:

- the price of the house and income level crime rate or school district
- predict growth and stock price
- employee performance and diplomas
- blood pressure levels and age,
- and more...

03*Linear regression types:

- Simple Linear Regression
- Multiple Linear Regression

Simple Linear Regression:

- The easiest model is the simple linear regression model with one input and one output.
- Based on the value of the single explanatory variable, the value of the response variable changes.
- The equation of a simple linear regression model to calculate the value of the dependent variable, y based on the predictor x, is as follows:

$$y_i = \beta_0 + \beta_i x + \epsilon$$

Y= Dependent Variable

X= Independent Variable

β_0 = intercept of the line

β_i = Linear regression coefficient (slope of the line)

ϵ = random error

Multiple Linear regression:

- The value of response variable is dependent upon more than one explanatory variables.
- Linear regression involving multiple variables is called "multiple linear regression."
- The steps to perform multiple linear regression are almost similar to that of a simple linear regression.
- The difference lies in the evaluation.
- One can use it to find out which factor has the highest impact on the predicted output and how different variables relate to each other.
- The equation of multiple linear regression is as follows:

$$y_i = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p + \epsilon$$

04*Coefficient & intercept:

The parameter β (the regression coefficient) signifies the amount by which change in x must be multiplied to give the corresponding average change in y, or the amount y changes for a unit increase in x.

- In this way it represents the degree to which the line slopes upwards or downwards.

We collect this data for 50 students in a certain college course and fit the following regression

model:

- Exam score = 65.4 + 2.67(hours)
- The value for the intercept term in this model is 65.4.
- This means the average exam score is 65.4 when the number of hours studied is equal to zero.
- This makes sense to interpret since it's plausible for a student to study for zero hours in preparation for an exam.

05*Assumptions for Multiple Linear regression:

- A linear relationship should exist between the target and predictor variables.
- MLR assumes little or no multicollinearity (correlation between the independent variable) in data.

06*Advantages/Disadvantages:

Advantages:

- Linear regression is one of the most interpretable machine learning algorithms. It is also easy to explain
- It is easy to use, as it requires minimal tuning.
- It is the most widely used machine learning technique.

Disadvantages:

- Model makes strong assumptions about the data.
- It works on only numeric features, so categorical data requires extra-processing.
- It does not do well with missing values and in the presence of outliers.

07*Linear regression:

```
#import data
import pandas as pd
stud_scores = pd.read_csv('student_scores.csv')
#EDA
stud_scores.head()
stud_scores.shape
stud_scores.describe()
stud_scores.info()
stud_scores.isna().sum()
stud_scores.corr()
stud_scores.head()
#Input and output
X = stud_scores.iloc[:, :-1].values # feature matrix
y = stud_scores.iloc[:, 1].values # response vector
#split data
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=0)
#Training the model
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
# coefficients of the trained model
print("\nCoefficient of model :", regressor.coef_)
# intercept of the model
print("\nIntercept of model", regressor.intercept_)
```

08*Fitting Comparison:

```
#Predictions
y_pred = regressor.predict(X_test)
#Comparing the test values and the predicted values:
compare_reg=pd.DataFrame({"Real":y_test,"Predicted": y_pred})
print(compare_reg)
```

09*Plotting:

```
#Plot Linear regression Python
import matplotlib.pyplot as plt
p=np.arange(0,max(y_pred)*1.1)
plt.plot(p,p,color='g')
plt.scatter(y_test, y_pred,color='b')
plt.show()
```

10*Multiple Linear regression:

Dataset 50_Startups.csv

How this model can help here?

•This machine learning model will be quite helpful in such a situation where we need to find a profit based on how much we are spending in the market and for the market. In a nutshell, this machine learning model will help to find out the profit based on the amount which we spend from the 50 startups dataset.

About the 50 startups dataset:

•This particular dataset holds data from 50 startups in New York, California, and Florida. The features in this dataset are R&D spending, Administration Spending, Marketing Spending, and location features, while the target variable is: Profit.

–Source.

1. R&D spending: The amount which startups are spending on Research and development.
2. Administration spending: The amount which startups are spending on the Admin panel.
3. Marketing spending: The amount which startups are spending on marketing strategies.
4. State: To which state that particular startup belongs.
5. Profit: How much profit that particular startup is making.

11*Multiple Linear regression with :

```
#import data
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
df=pd.read_csv('50_Startups.csv')
df.head()
df.shape
df.describe()
df.info()
df.corr()
df.dropna()
print(df.head())
df = pd.get_dummies(df)
print(df.head())
#input & output
X=df.drop(columns=['Profit'])
y=df['Profit']
#split data to training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.30, random_state=0)
#Training the model
print(X_train.shape)
print(y_train.shape)
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
# coefficients of the trained model
print("\nCoefficient of model :", linear_model.coef_)
# intercept of the model
print("\nIntercept of model',linear_model.intercept_)
# How good is the fit
y_pred = linear_model.predict(X_test)
plt.scatter(y_test, y_pred,color='b')
```

```
plt.xlabel("actual")
plt.ylabel("predicted")
plt.show()
```

12*Linear regression in ML:

Regression searches for relationships among variables.

- For example, you can observe several employees of some company and try to understand how their salaries depend on their features, such as experience, education level, role, city of employment, and so on.
- This is a regression problem where data related to each employee represents one observation.
- The presumption is that the experience, education, role, and city are the independent features, while the salary depends on them.

13*Logistic regression:

- The name of this algorithm can be a little confusing in the sense that logistic regression algorithm is used for classification tasks.
- In fact, logistic regression is much similar to the linear regression except that how it is used.
- Linear regression is used for regression problems, whereas logistic regression is used for solving the classification problems.
- The name “Regression” implies that a linear model is to fit into the linear space.
- Predictions are mapped to be between 0 and 1 through the logistic function shown in Fig. 2.6 to a linear combination of features [16],
- Logistic regression is a classic predictive modeling technique and still remains a popular choice for modeling binary categorical variables.
- Logistic regression is the classification counterpart of linear regression.
- It is also one of the most popular supervised machine learning algorithms.
- It is used to predict the categorical dependent variable using a given set of independent variables. Logistic regression is an efficient and powerful way to analyze the effects of a group of independent variables with binary outcomes by quantifying each independent variable’s unique contribution to predict the output of a categorical dependent variable.
- Therefore, the outcomes must be categorical or should have discrete values.
- Each can be either yes or no, 0 or 1, true or false, etc., but instead of giving the exact value as 0 or 1, it gives the probabilistic values which lie between 0 and 1.
- This method can be used in spam detection, credit card fraud, predicting whether a given mass of tissue is benign or malignant, etc

Chapter 05 : Naïve Bayes

01*Naïve Bayes classifier:

- Naïve Bayes classifiers are linear classifiers that are known for being simple yet very efficient.
- Naïve Bayes (NB) classifier is among the most popular learning method grouped by similarities that works on the popular Bayes Theorem of probability with an assumption of independence between predictors—to build ML models.
- Along with simplicity, Naïve Bayes is known to outperform even highly sophisticated classification methods.
- The method has a strong assumption of independent input variables which is not always realistic for real data.
- However, strong violations of the independent assumptions and nonlinear classification problems may lead to poor performances of

02*Naïve Bayes classifiers:

Along with simplicity, Naïve Bayes is known to outperform even highly sophisticated classification methods.

- The method has a strong assumption of independent input variables which is not always realistic for real data.
- However, strong violations of the independent assumptions and nonlinear classification problems may lead to poor performances of
- It is called naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features.
- Such as if the fruit is to be identified on the basis of color, shape, and taste, then a yellow, spherical, and sweet fruit is recognized as an orange.
- Hence, each feature individually contributes to identify that it is an orange without depending on each other.
- It is called Bayes' because it depends on the principle of Bayes' Theorem.

03*Bayes' Theorem:

Bayes' theorem is also known as Bayes' rule or Bayes' law, which is used to determine the probability of a hypothesis with prior knowledge.

- It depends on the conditional probability.
- The formula for Bayes' theorem is given as:

$$P(A|B) = P(B|A) / P(B)$$

where

- $P(A|B)$ is posterior probability. Probability of hypothesis A on the observed event B.
 - $P(B|A)$ is likelihood probability. Probability of the evidence B given that the probability of a hypothesis A is true.
 - $P(A)$ is prior probability. Probability of hypothesis before observing the evidence.
 - $P(B)$ is marginal probability. Probability of evidence.
- $P(H|E)$ denotes how event H happens when event E takes place.
- $P(E|H)$ represents how often event E happens when event H takes place first.
 - $P(H)$ represents the probability of event X happening on its own.
 - $P(E)$ represents the probability of event Y happening on its own.

04*Types of Naïve Bayes Model:

Gaussian:

- The Gaussian model assumes that features follow a normal distribution.
- This means if predictors take continuous values instead of discrete, then the
- model assumes that these values are sampled from the Gaussian distribution.

Multinomial:

- The multinomial Naïve Bayes classifier is used when the
- data is multinomial distributed. It is primarily used for document classification problems.
- It means a particular document belongs to which category such as politics, sports, and entertainment.
- The features/predictors used by the classifier are the frequency of the words present in the document.

Bernoulli:

The Bernoulli classifier works similar to the multinomial classifier, but the predictor variables are the independent Boolean variables, for example, if a particular word is present or not in a document.

04*Assumptions:

- The fundamental Naïve Bayes assumption is that each feature makes an independent and equal (i.e., identical) contribution to the outcomes

05*How Naïve Bayes Algorithm Works?

An NB model comprises two types of probabilities that can be calculated directly from the training data:

- 1.The probability of each class
 - 2.The conditional probability for each class given each input value
- A Naïve Bayes model multiplies several different calculated probabilities together to identify the posterior probability for each class and select the class with the highest probability.
 - This is called the maximum a posteriori probability.

Conditional independence of the features reduces the complexity of model

Following steps are followed while working with this algorithm:

- 1.Calculate prior probability for the given class labels.
 - 2.Calculate conditional probability with each attribute for each class.
 - 3.Multiply same class conditional probability.
 - 4.Multiply prior probability with probability obtained in step 3.
 - 5.Determine which class has a higher probability.
- Higher probability class belongs to the given input set.

A training data set of weather and corresponding target variable 'Play'.

- 14 days Observations(suggesting possibilities of playing).
- We need to classify whether players will play or not based on weather condition.
- 2 Classes YES and NO

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes
Overcast	Yes
Rainy	No

Step 1: Convert the data into a frequency table

Weather	No	Yes
Overcast		4
Rainy	3	2
Sunny	2	3
Grand Total	5	9

Step 2: Create Likelihood table by finding the probabilities

Weather	No	Yes		
Overcast		4	4/14	0.29
Rainy	3	2	5/14	0.36
Sunny	2	3	5/14	0.36
Grand Total	5	9		
	5/14	9/14		
	0.36	0.64		

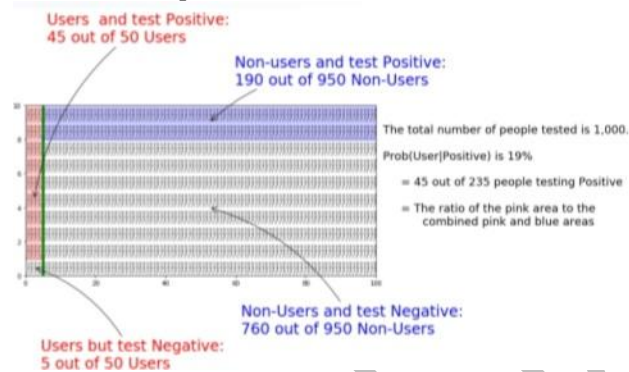
Step 3: Now, use Naive Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of prediction.

06* True positive rate / True negative rate:

		Actual	
		Positive	Negative
Predicted	Positive	True Positive Rule Matched and Attack Present	False Positive Rule Matched and No Attack Present
	Negative	False Negative No Rule Matched and No Attack Present	True Negative No Rule Matched and Attack Present

Weather	Play
Sunny	No
Overcast	Yes
Rainy	Yes
Sunny	Yes
Sunny	Yes
Overcast	Yes
Rainy	No
Rainy	No
Sunny	Yes
Rainy	Yes
Sunny	No
Overcast	Yes

07* Positive predictive value:



08* Bayesian Probabilities:

Another form of Bayes' theorem for two competing statements or hypotheses is:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B|A)P(A) + P(B|\neg A)P(\neg A)}$$

$$P(\text{Yes}|\text{Sunny}) = \frac{P(\text{Sunny}|\text{yes}) \cdot P(\text{yes})}{P(\text{Sunny}|\text{yes}) \cdot P(\text{yes}) + P(\text{Sunny}|\text{No}) \cdot P(\text{No})}$$

$$P(\text{No}|\text{Sunny}) = \frac{P(\text{Sunny}|\text{No}) \cdot P(\text{No})}{P(\text{Sunny}|\text{No}) \cdot P(\text{No}) + P(\text{Sunny}|\text{Yes}) \cdot P(\text{yes})}$$

09*Prediction using Bayesian equation:

Players will play or not(Target) if weather is sunny (Attribute).

$$P(\text{Sunny}) = 5/14 = 0.36$$

$$P(\text{Yes}) = 9/14 = 0.64$$

$$P(\text{No}) = 5/14 = 0.36$$

$$P(\text{Sunny} | \text{Yes}) = 3/9 = 0.33$$

$$P(\text{Sunny} | \text{No}) = 2/5 = 0.4$$

$$P(\text{Sunny} | \text{yes}) * P(\text{yes}) / P(\text{Sunny}) = 0.33 * 0.64 / 0.36 = 0.59 \text{ class with a higher probability}$$

$$P(\text{Sunny} | \text{No}) * P(\text{No}) / P(\text{Sunny}) = 0.4 * 0.36 / 0.36 = 0.4$$

Players will play or not(Target)

If weather is sunny (Attribute).

$$P(\text{Sunny}) = 5/14 = 0.36$$

$$P(\text{Yes}) = 9/14 = 0.64$$

$$P(\text{No}) = 5/14 = 0.36$$

$$P(\text{Sunny} | \text{Yes}) = 3/9 = 0.33$$

$$P(\text{Sunny} | \text{No}) = 2/5 = 0.4$$

$$P(\text{Sunny} | \text{yes}) * P(\text{yes}) = 0.33 * 0.64 = 0.211$$

$$P(\text{Sunny} | \text{No}) * P(\text{No}) = 0.4 * 0.36 = 0.144$$

$$P(\text{Yes} | \text{Sunny}) = 0.211 / (0.211 + 0.144) = 0.59 \text{ class with a higher probability}$$

$$P(\text{No} | \text{Sunny}) = 0.144 / (0.144 + 0.211) = 0.41$$

Weather	No	Yes		
Overcast		4	4/14	0.29
Rainy	3	2	5/14	0.36
Sunny	2	3	5/14	0.36
Grand Total	5	9		
	5/14	9/14		
	0.36	0.64		

10*Naïve Bayes with multiple predictors:

	Outlook	Temp	Humidity	Windy	Play
0	Rainy	Hot	High	f	no
1	Rainy	Hot	High	t	no
2	Overcast	Hot	High	f	yes
3	Sunny	Mild	High	f	yes
4	Sunny	Cool	Normal	f	yes
5	Sunny	Cool	Normal	t	no
6	Overcast	Cool	Normal	t	yes
7	Rainy	Mild	High	f	no
8	Rainy	Cool	Normal	f	yes
9	Sunny	Mild	Normal	f	yes
10	Rainy	Mild	Normal	t	yes

• test= 13 Sunny Mild High t

$$P(\text{yes} | \text{test}) = \frac{P(\text{yes}) * P(\text{Sunny} | \text{yes}) * P(\text{mild} | \text{yes}) * P(\text{high} | \text{yes}) * P(\text{t} | \text{yes})}{P(\text{Sunny}) * P(\text{mild}) * P(\text{high}) * P(\text{t})}$$

$$P(\text{No} | \text{test}) = \frac{P(\text{No}) * P(\text{Sunny} | \text{No}) * P(\text{mild} | \text{No}) * P(\text{high} | \text{No}) * P(\text{t} | \text{No})}{P(\text{Sunny}) * P(\text{mild}) * P(\text{high}) * P(\text{t})}$$

11*Applications of Naïve Bayes:

Real time Prediction (Traveling ..)

- Multi class Prediction
- Text classification
- Spam Filtering
- Sentiment Analysis
- Recommendation System (Recommend a product to another client)

12*Advantages/Disadvantages:

Advantages

- NB models performs well in practice even when conditional independence assumption rarely holds true.
- It is easy to implement and can scale with data.
- It is a good choice when CPU and memory resources are a limiting factor.
- It performs well when the input variables are categorical.
- It is easier to predict class of the test data set.

Disadvantages

- NB assumes that all features are independent or unrelated, so it cannot learn the relationship between features.

13*Dataset playsheet dataset:

	Outlook	Temp	Humidity	Windy	Play
0	Rainy	Hot	High	f	no
1	Rainy	Hot	High	t	no
2	Overcast	Hot	High	f	yes
3	Sunny	Mild	High	f	yes
4	Sunny	Cool	Normal	f	yes
5	Sunny	Cool	Normal	t	no
6	Overcast	Cool	Normal	t	yes
7	Rainy	Mild	High	f	no
8	Rainy	Cool	Normal	f	yes
9	Sunny	Mild	Normal	f	yes

14*Sample algorithm with python:

NaiveBayes project (Weather Prediction)

```
#Required Modules
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
df=pd.read_csv('playsheet_dataset.csv')
inputs=df.drop('Play',axis='columns')
target=df['Play']
Numerics=LabelEncoder()
inputs['outlook_n']=Numerics.fit_transform(inputs['Outlook'])
inputs['Temp_n']=Numerics.fit_transform(inputs['Temp'])
inputs['Humidity_n']=Numerics.fit_transform(inputs['Humidity'])
inputs['windy_n']=Numerics.fit_transform(inputs['Windy'])
inputs_n=inputs.drop(['Outlook','Temp','Humidity','Windy'],axis='columns')
Classifier=GaussianNB()
Classifier.fit(inputs_n,target)
Classifier.predict([[0,0,0,1]]) #array(['yes'], dtype='<U3')
```

15*One hot Encoder:

	Outlook	Temp	Humidity	Windy	Play
0	Rainy	Hot	High	f	no
1	Rainy	Hot	High	t	no
2	Overcast	Hot	High	f	yes
3	Sunny	Mild	High	f	yes
4	Sunny	Cool	Normal	f	yes

Before

	outlook_n	Temp_n	Humidity_n	windy_n
0	1	1	0	0
1	1	1	0	1
2	0	1	0	0
3	2	2	0	0
4	2	0	1	0

After

Chapter 06 : KNearestNeighbors(KNN)

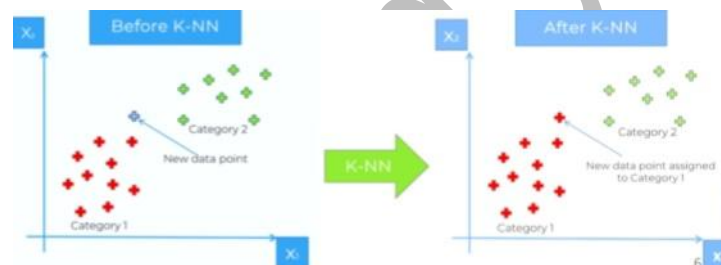
01*What is KNN ?

- KNN is a very simple and effective machine learning algorithm.
- It is a non-parametric, lazy-learning algorithm, which means that there is no explicit training phase before classification.
- KNN algorithm uses the entire dataset as the training set, i.e., it does not split the dataset into training and testing sets.
- KNN algorithm assumes that similar entities exist in close proximity.
- In other words, similar entities have features that are close to each other.

K-nearest neighbors (KNN) algorithm is a type of supervised ML algorithm which can be used for

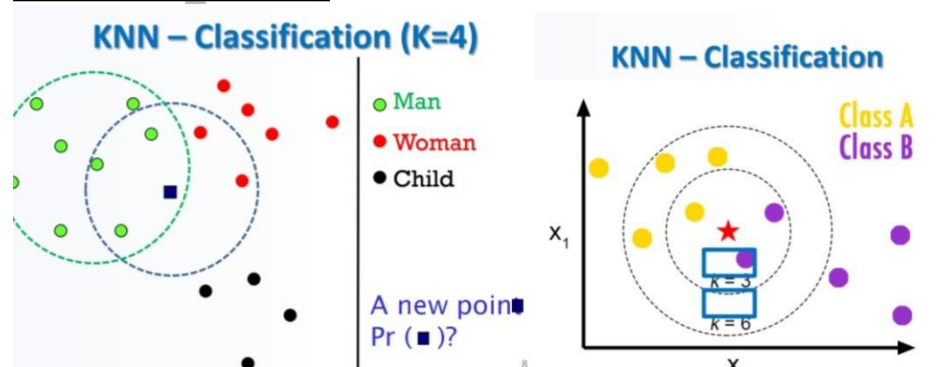
- Classification,
- Regression
- or outliers detection problems.

"Similar things are near to each other".

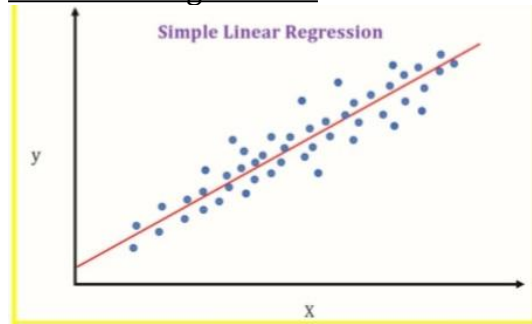


KNN uses 'feature similarity' to predict the value of new data point – based on how closely it matches the points in the training set.

02*KNN Classification:

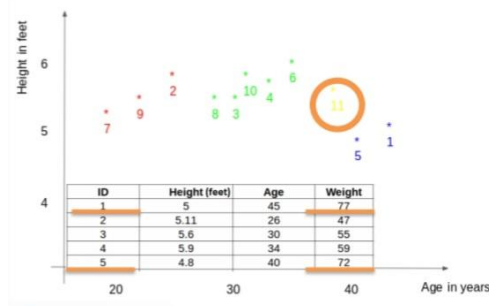


03*KNN – Regression:



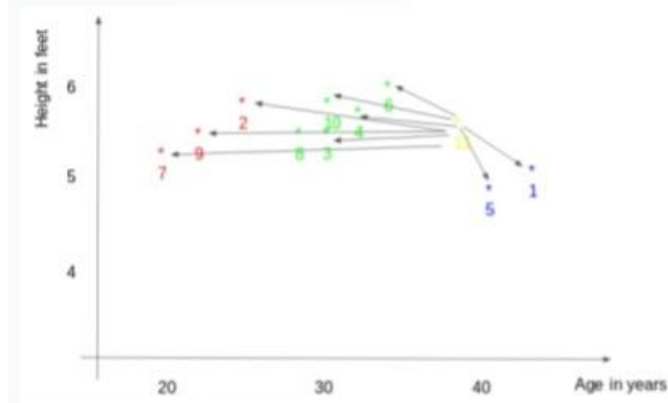
ID	Height (feet)	Age	Weight
1	5	45	77
2	5.11	26	47
3	5.6	30	55
4	5.9	34	59
5	4.8	40	72
6	5.8	36	60
7	5.3	19	40
8	5.8	28	60
9	5.5	23	45
10	5.6	32	58
11	5.5	38	?

- What is the weight of ID11 based on the plot?
- ID11 is closer to points 5 and 1
- The weight is probably between 72 and 77 kg

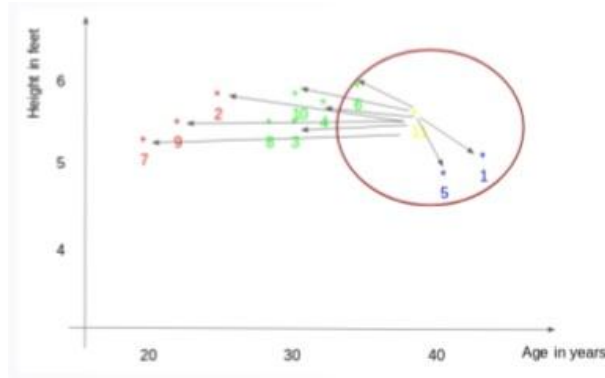


04*KNN – Regression Steps:

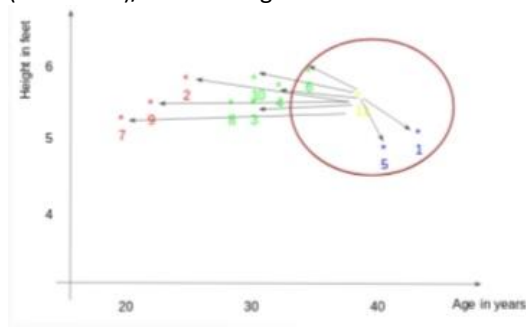
- 1.First, calculate the distance between the new point and each training point.



2. The closest k data points are selected (based on the distance). The points 1, 5, 6 will be selected if k = 3.



3. The average of these data points is the prediction for the new point. The weight of ID11 = $(77+72+60)/3 = 69.66$ kg.



04*KNN Applications:

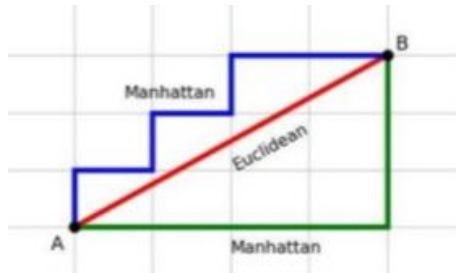
- Businesses:** To understand the relationship between advertising spending and revenue.
- Medical researchers:** To understand the relationship between drug dosage and blood pressure of patients.
- Agricultural scientists:** To measure the effect of fertilizer and water on crop yields.
- Data scientists for professional sports teams:** To measure the effect that different training regimens have on player performance.
- Stock predictions:** To predict how stocks will perform in the future.
- Consumer behavior:** How much a customer is likely to spend.

05*How Does KNN Works?

- 1) Select the K where K is the number of neighbors.
- 2) Compute the distance between test point and each point.
- 3) Sort the distances in ascending (or descending) order.
- 4) Use the sorted distances to select the K-nearest neighbors.
- 5) Use majority rule (for classification) or averaging (for regression) to assign the class or value, respectively.

06*Calculating the distance:

- There are various methods for calculating this distance:



Euclidean :

Euclidean $\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$

Manhattan:

Manhattan $\sum_{i=1}^k |x_i - y_i|$

Hamming :

$$D_H = \sum_{i=1}^k |x_i - y_i|$$

$$x = y \Rightarrow D = 0$$

$$x \neq y \Rightarrow D = 1$$

How to choose the k factor?

- The second step is to select the k value.
- This determines the number of neighbors we look at when we assign a value to any new observation.
- In our example, $k = 3$, the closest points are ID1, ID5 and ID6.
- The prediction of weight for ID11: $(77+72+60)/3 = 69.66$ kg

ID	Height	Age	Weight
1	5	45	77
5	4.8	40	72
6	5.8	36	60

For the value of $k=5$, the closest point will be ID1, ID4, ID5, ID6 and ID10.

The prediction for ID11: $(77+59+72+60+58)/5 = 65.2$ kg

ID	Height	Age	Weight
1	5	45	77
4	5.9	34	59
5	4.8	40	72
6	5.8	36	60
10	5.6	32	58

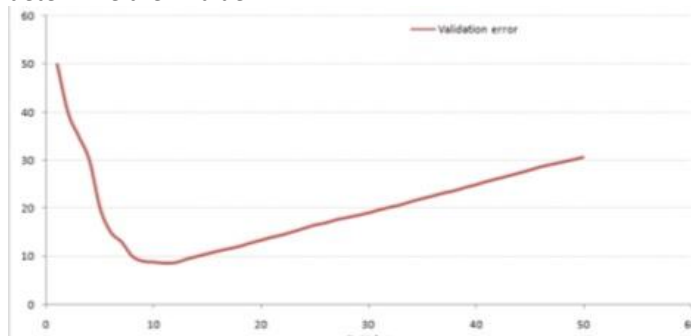
- There is no particular way of choosing the value K, but here are some common conventions to keep in mind:
 - Choosing a very low value will most likely lead to inaccurate predictions.
 - The commonly used value of K is 5.
 - Always use an odd number as the value of K.

How can we figure out the optimum value of k?

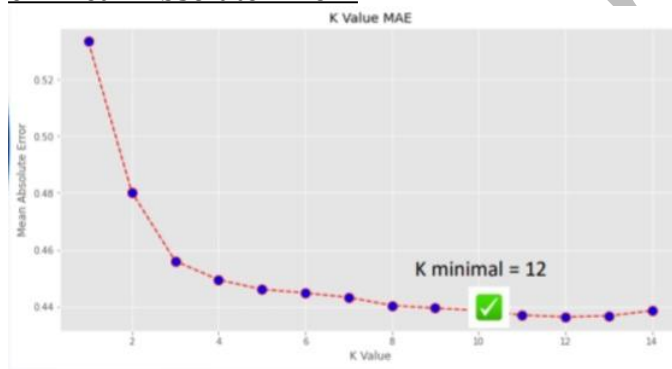
- Let us decide it based on the error calculation for our train and validation set –after all, minimizing the error is our final goal!.
- Have a look at the below graphs for training error and validation error for different values of k.

Let us decide it based on the error calculation for our train and validation set (after all, minimizing the error is our final goal!).

- Have a look at the below graphs for training error and validation error for different values of k. For a very low value of k (suppose k=1), the model overfits on the training data, which leads to a high error rate on the validation set.
- For a high value of k, the model performs poorly on both train and validation set.
- If you observe closely, the validation error curve reaches a minima at a value of k = 9.
- This value of k is the optimum value of the model (it will vary for different datasets).
- This curve is known as an 'elbow curve' (because it has a shape like an elbow) and is usually used to determine the k value.



07*Mean Absolute Error:



08*KNN Classification example:

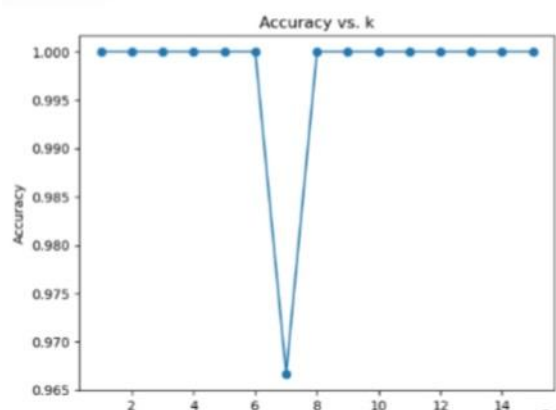
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target
# Convert the dataset into a pandas DataFrame for exploration
iris_df = pd.DataFrame(X, columns=iris.feature_names)
iris_df['target'] = y
# Data exploration using pandas
print("Iris Dataset:")
print(iris_df.head())
```

```

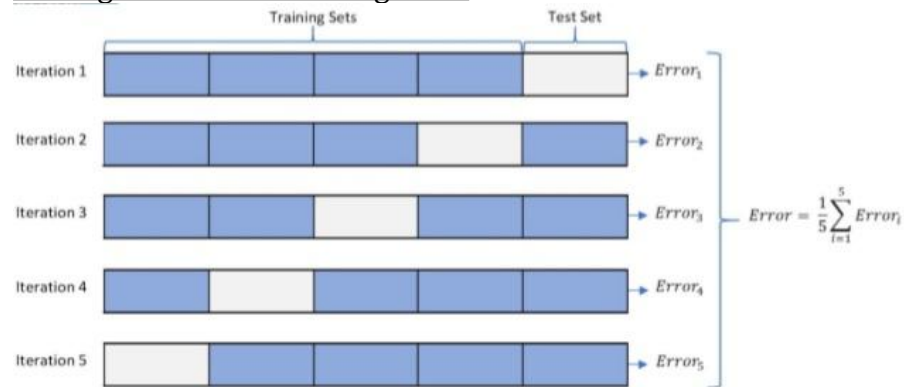
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
# Find the optimal value of k
max_k = min(15, len(X_train)) # Limit the maximum value of k to avoid overfitting
best_accuracy = 0
best_k = 0
accuracy_values = []
k_values = []
for k in range(1, max_k+1):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    accuracy_values.append(accuracy)
    k_values.append(k)
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        best_k = k
# Train the KNN classifier using the optimal value of k
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print('accuracy score: {}'.format(round(accuracy*100,2)))
# Plot the results
plt.plot(k_values, accuracy_values, marker='o')
plt.xlabel('k')
plt.ylabel('Accuracy')
plt.title('Accuracy vs. k')
plt.show()

```

Accuracy score :96.67%



09*Regression Model Using KNN:



10*Advantages:

- 1.No Training Period: KNN is called Lazy Learner. It does not learn anything in the training period. This makes the KNN algorithm much faster than other algorithms that require training e.g. Linear Regression etc.
- 2.Since the KNN algorithm requires no training before making predictions, new data can be added seamlessly which will not impact the accuracy of the algorithm.
- 3.KNN is very easy to implement. There are only two parameters required to implement KNN i.e. the value of K and the distance function (e.g. Euclidean or Manhattan etc.)

11*Disadvantages:

- 1.Does not work well with large dataset: In large datasets, the cost of calculating the distance between the new point and each existing points is huge which degrades the performance of the algorithm.
- 2.Does not work well with high dimensions: The KNN algorithm doesn't work well with high dimensional data because with large number of dimensions, it becomes difficult for the algorithm to calculate the distance in each dimension.
- 3.Need feature scaling: We need to do feature scaling(standardization and normalization) before applying KNN algorithm to any dataset. If we don't do so, KNN may generate wrong predictions.
- 4.Sensitive to noisy data, missing values and outliers: KNN is sensitive to noise in the dataset. We need to manually impute missing values and remove outliers.

12*Generalization in ML:

- Machine learning can be described as the process of "generalizing from examples".
- Generalization in machine learning is used to measure the model's performance to classify unseen data samples.
- A model is said to be generalizing well if it can forecast data samples from diversified sets.
- The model should be fitted correctly.
- Neither over-fitting nor under-fitting should occur

13*Some key terms:

❑ **Statistical fit:** The goodness of the fit is the statistical technique to measure how well the data points fit the model's curve. The model is a good fit if it converges well between the expected and observed data points.

❑ **Training error:** The fitness of the predictive model is calculated by comparing the actual and expected values. When this is done on the training data itself, it gives us the training error.

❑ **Test error:** Predictive models find the mapping between the input variable and the target value. The corresponding error is called test error when we test the model on an unknown dataset.

❑ **Bias:** It measures the difference between the model's prediction and the target value.

If the model is oversimplified, the predicted value would be far from the ground truth resulting in more bias.

Variance: Variance measures the inconsistency of different predictions over a varied dataset. Suppose the model's performance is tested on different datasets—the closer the prediction, the lesser the variance. Higher variance indicates overfitting, in which the model loses the ability to generalize.

14*How good the fit is:

- This best fit line is divided into 3 types depending on the precision level- Underfit, Good fit and Overfit.

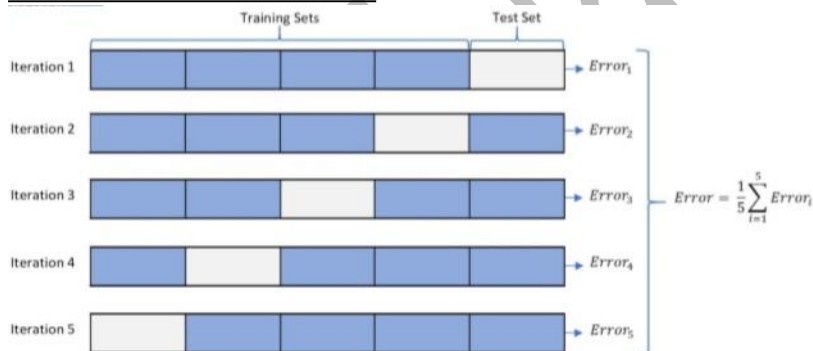
15*Overfitting:

- Over-fitting is a modeling error, which occurs if a function is defined to closely fit to a limited set of data points during the training phase.
- It makes a model conform to mimic a dataset that may not be fully representative of other data points that the model may encounter in future.
- One can tell a model is overfitting when it performs great on training set, but poorly on test set (or new data).
- A statistical model is said to be overfitted when the model does not make accurate predictions on testing data.
- The error produced from the training dataset is known as Bias and the error by testing data set is Variance.
- The aim of any model will be to obtain a low bias and low variance model.

16*Overfitting Solution?

- K-fold cross-validation is one of the most common techniques used to detect overfitting.
- Here, we split the data points into k equally sized subsets in K-folds cross-validation, called "folds."
- One split subset acts as the testing set while the remaining groups are used to train the model

17*K-fold cross-validation:



18*Overfitting happens when:

- The training data is not cleaned and contains some "garbage" values. The model captures the noise in the training data and fails to generalize the model's learning.
- The model has a high variance.
- The training data size is insufficient, and the model trains on the limited training data for several epochs.
- The architecture of the model has several neural layers bundled together. Deep neural networks are complex and require a significant amount of time to train, and often lead to overfitting the training set.
- Incorrect tuning of hyperparameters in the training phase leads to over-observing the training set, resulting in memorizing features.

19*Underfitting:

- A statistical model or a machine learning algorithm is said to have underfitting when a model is too simple to capture data complexities.
- It represents the inability of the model to learn the training data effectively result in poor performance both on the training and testing data.
- In simple terms, an underfit model's are inaccurate, especially when applied to new, unseen examples.
- It mainly happens when we use very simple model with overly simplified assumptions.
- To address underfitting problem of the model, we need to use more complex models, with enhanced feature representation, and less regularization.
- The underfitting model has High bias and low variance. Unclean training data containing noise or outliers can be a reason for the model not being able to derive patterns from the dataset.
- ❑ The model has a high bias due to the inability to capture the relationship between the input examples and the target values. This usually happens in the case of varied datasets.
- ❑ The model is assumed to be too simple—for example, we train a linear model in complex scenarios.
- ❑ Incorrect hyperparameters tuning often leads to underfitting due to under-observing of the features.

20*Techniques to Reduce Underfitting:

- Increase model complexity.
- Increase the number of features, performing feature engineering.
- Remove noise from the data.
- Increase the number of epochs or increase the duration of training to get better results.

21*How to achieve a Good Fit:

1. Introduction of the validation set
2. Resampling methods

22*Validation set:

- A set of observations used during model training to provide feedback on how well the current parameters generalize beyond the training set.
- If training error decreases but validation error increases, the model is likely overfitting and one should pause training.
- Bias is an error from erroneous assumptions in the learning algorithm.
- High bias can cause an algorithm to miss the relevant relations between features and target outputs (under fitting).

23*How to avoid Overfitting?

1. Train with more data
2. Data augmentation
3. Addition of noise to the input data
4. Feature selection
5. Cross-validation
6. Simplify data
7. Regularization
8. Ensembling
9. Early stopping

24*How to reduce Underfitting?

1. Decrease regularization
2. Increase the duration of training
3. Feature selection
4. Remove noise from data

25*Overfitting vs Underfitting:

- ❑ Overfitting and Underfitting are the two biggest causes of the poor performance of machine learning algorithms and models.
- ❑ The scenario in which the model performs well in the training phase but gives a poor accuracy in the test dataset is called overfitting.
- ❑ The scenario in which the model performs poorly on the training dataset as it cannot derive features from the training set.
- ❑ To achieve a good fit means to stop training the model at an optimal point such that the model is neither under observing the features nor learning the unnecessary details and the noise in the training set.

Overfitting: Too Much of a Good Thing

• Imagine you're studying for a test. You go through your notes, but then you decide to memorize every tiny detail, including the doodles in the margins. When the test comes, you're ready for anything... except the actual questions. This is overfitting.

Underfitting: Falling Short

• Now, picture yourself trying to bake a cake. You have the recipe, but you decide to skip some crucial steps because you're feeling lazy. When the cake comes out, it's a disaster — half-baked and unappetizing. This is underfitting.

26*Overfitting & Underfitting:

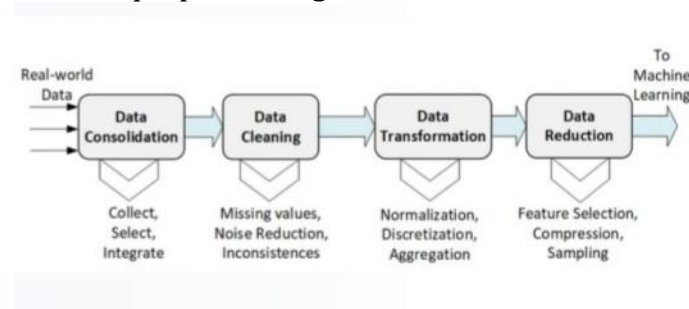
- In Underfit, the best fit line doesn't cover many data points present. Thus, in the training dataset itself, there is a high chance of occurrence of error (high bias). And in the testing dataset (low variance).
- In an overfit line, the best fit line covers every single data point. You might think isn't that what we want?? But no, this may be well and good in the case of training dataset (low bias). But, when a testing dataset is provided to the same model, there will be a high error in the testing dataset (high variance).
- But only a Good fit line, the best fit line will be in such a way that any point to be predicted will be accurately predicted. Consequently, it will still have a low bias and low variance.

27*Importance of Feature Selection:

- Machine learning models work on a simple rule—garbage in, garbage out.
- By garbage here means noise in data.
- The high-dimensional dataset can lead to a lot of problems, namely long training time, model could be complex which in turn may lead to over-fitting.
- We do not need to use every feature that is present in the dataset.
- One can assist modeling by feeding in only those features that are really important.
- Sometimes, less is better
- In a case with very high dimensions, there are several redundant features that do not contribute much but are simply extensions of the other important features.
- These redundant features do not contribute to the model's predictive capability.
- Clearly, there is a need to remove these redundant features from the dataset to get the most effective predictive modeling performance.
- We can summarize the objectives of feature selection as follows:
 - It enables faster training of a model.
 - It reduces the complexity of the model, and it becomes easier to interpret and avoids over fitting.
 - It improves the prediction accuracy of a model.
 - Less data to store and process.

Chapter 07 : Feature Scaling Normalization Standardization

01*Data preprocessing:



02*What is the need for data clean-up before feeding it to ML algorithms?

- Goal of data clean-up is to identify and remove any duplicate or erroneous data elements, to create a reliable dataset.
- This improves the quality of input data for the ML model training.
- It results in accurate decision-making during ML inference phase.

03*Exploratory Data Analysis EDA:

- EDA is a phase used for data science pipeline in which the focus is to understand insights of the data through visualization or by statistical analysis:
 - Multicollinearity
 - Duplicates values
 - Null Values.....

04*Duplicated Values:

- A dataset may include data objects, which are duplicates of one another.
- For example, it may happen when the same person submits a form more than once.
- In this case, we may have repeated information about the person's name or email address.
- Duplicates may give advantage or bias to the particular data.
- Therefore, they need to be removed

05*Data Transformation:

The selected and preprocessed data is transformed using one or more of the following methods:

- Aggregation:** This is to collate a bunch of data features into a single one.
- Scaling:** It involves selecting the right feature scaling for the selected and preprocessed data

06*Feature Aggregation Example:

- Feature aggregation is performed so as to take aggregated values in order to put the data in a better perspective.
- Consider the data on electricity usage over a day.
- Aggregating the usage on an hourly or daily basis will help in reducing the number of observations.
- This results in saving memory and reducing processing time.
- Aggregations provide with a high-level view of the data as the behavior of groups or aggregates is more stable than individual data objects

07*Feature scaling:

- Feature scaling is an important step in the data transformation stage of the data preparation process.
- It is the method to limit the range of variables so that they can be compared on common grounds..
- It means to adjust values of numeric features measured on different scales to a notionally common scale, without altering differences in the value's ranges or losing information.
- This is the last step involved in Data Preprocessing and before ML model training.
- We apply Feature Scaling on independent variables.
- We fit feature scaling with train data and transform on train and test data.
- The goal is to improve the overall quality of the dataset by re-scaling the dimension of the data and avoiding situations in which some values over-weighting others

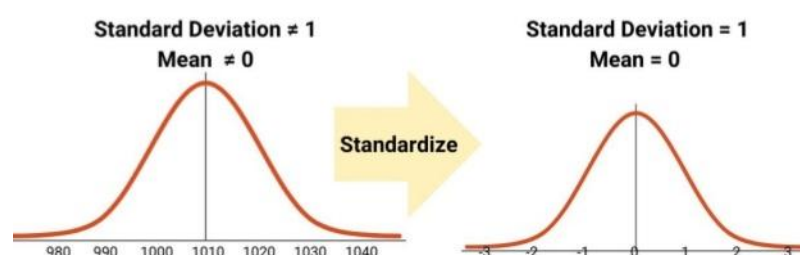
08*Why is feature scaling necessary?

- ML algorithms are sensitive to the relative scale of features, which are the input values used for training a model.
- Feature scaling is needed for ML algorithms that calculate and use distances between data. Examples of such algorithms include KNN, K-Means, etc.
- If input data is not scaled or normalized, a feature with higher values will dominate ML parameter settings and thus skew the model results.

09*Feature scaling:

- Many machine learning algorithms perform better when numerical input variables are scaled to a standard range.
- This includes algorithms that use a weighted sum of the input, like linear regression, and algorithms that use distance measures, like k-nearest neighbors.
- The two most popular techniques of Feature Scaling are:
 - Standardization
 - Normalization
- Both normalization and standardization can be achieved using the scikit-learn library.

10*Feature Scaling, Standardization & Normalization:



11*Standardization:

- Standardizing, a dataset involves rescaling the distribution of values so that
 - the mean of observed values is 0
 - and the standard deviation is 1.

$$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation}(x)}$$

- A standard deviation (or σ) is a measure of how dispersed the data is in relation to the mean.
- Low, or small, standard deviation indicates data are clustered tightly around the mean, and high, or large, standard deviation indicates data are more spread out.

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

σ = population standard deviation
 N = the size of the population
 x_i = each value from the population

$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation}(x)}$

#	Emp	Age	Salary
1	Emp1	44	73000
2	Emp2	27	47000
3	Emp3	30	53000
4	Emp4	38	62000
5	Emp5	40	57000
6	Emp6	35	53000
7	Emp7	48	78000

Mean = 37.42857 Mean = 60428.5714
 Std. Dev. = 6.883876 Std. Dev. = 10499.7570

How to calculate Standardized value?
 = 35, mean = 37.42, Std. Dev. = 6.88
 for column Age:
 $x_{\text{stand}}(\text{for } 35) = \frac{35 - 37.42}{6.88} = -0.3527$

Age	Standardized Age	Salary	Standardized Salary
44	0.954611636	73000	1.197306616
27	-1.514927162	47000	-1.278941158
30	-1.079126198	53000	-0.707499364
38	0.083009708	62000	0.149663327
40	0.373543684	57000	-0.326538168
35	-0.352791257	53000	-0.707499364
48	1.535679589	78000	1.673508111

Mean = 0 Std. dev. = 1 Mean = 0 Std. dev. = 1

12*Normalization:

- Normalization is the process of rescaling data so that they have the same scale typically in the interval [1,+1] or [0,1].
- Normalization is used to avoid over-fitting and improving computation speed.
- Normalization is used when the attributes in data have varying scales.
- Normalization refers to rescaling the feature between min and max (usually between 0 and 1).
- To normalize the feature, subtract the min value from each feature instance and divide by the range of the feature (max-min) as shown below.
- Sometimes machine models are not based on Euclidean distances (ED), we will still need to do features scaling for the algorithm to converge much faster.
- If we skip feature scaling, then the models will run for a longer time.

$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$

#	Emp	Age	Salary
1	Emp1	44	73000
2	Emp2	27	47000
3	Emp3	30	53000
4	Emp4	38	62000
5	Emp5	40	57000
6	Emp6	35	53000
7	Emp7	48	78000

Normalization

Age	Normalized Age	Salary	Normalized Salary
44	0.80952381	73000	0.838709677
27	0	47000	0
30	0.142857143	53000	0.193548387
38	0.523809524	62000	0.483870968
40	0.619047619	57000	0.322580645
35	0.380952381	53000	0.193548387
48	1	78000	1

Range 0-1 Range 0-1

How to calculate Normalized value?
 $X = 35, \min = 27, \max = 48$ for column Age.
 $x_{\text{norm}}(\text{for } 35) = \frac{35 - 27}{48 - 27} = 0.3809$

13*Which Required Feature Scaling?

Those Algorithms Calculate Distance

- ☐ K-Means
- ☐ K-Nearest Neighbors (KNN)
- ☐ Linear Discriminant Analysis
- ☐ Support Vector Machine (SVM)
- ☐ Principal Component Analysis (PCA)

Gradient Descent Based Algorithms

- Linear Regression
- Logistic Regression
- Neural Network

14*Types of Feature Scaling:

- Min Max Scaler
- Standard Scaler
- Max Abs Scaler
- Robust Scaler
- Unit Vector Scaler
- Power Transformer Scaler
- Quantile Transformer Scaler

15*Using StandardScaler () from sklearn:

```
from sklearn.preprocessing import StandardScaler
data = [[0, 61],[0, 62],[1, 63],[1, 64]]
s = StandardScaler()
s.fit_transform (data)
```

```
array([[ -1.          , -1.34164079],
       [ -1.          , -0.4472136 ],
       [  1.          ,  0.4472136 ],
       [  1.          ,  1.34164079]])
```

Using normalize() from sklearn:

```
from sklearn.preprocessing import Normalizer
data = [[0, 61],[0, 62],[1, 63],[1,64]]
s = Normalizer()
s.fit_transform (data)
```

```
array([[0.          , 1.          ],
       [0.          , 1.          ],
       [0.00158478, 0.99999874],
       [0.01562309, 0.99987795]])
```

Using MinMaxScaler () from sklearn:

```
from sklearn.preprocessing import MinMaxScaler
data = [[0, 61],[0, 62],[1, 63],[1, 64]]
s = MinMaxScaler()
s.fit_transform (data)
```

```
array([[0.      , 0.      ],
       [0.      , 0.33333333],
       [1.      , 0.66666667],
       [1.      , 1.      ]])
```

16*MinMaxScaler/Normalizer:

- MinMaxScaler is applied column-wise, Normalizer is applied row-wise.
- In both cases, you're transforming the values of numeric variables so that the transformed data points have specific helpful properties.
- The difference is that:
 - in scaling, you're changing the range of your data,
 - in normalization, you're changing the shape of the distribution of your data.

17*A KNN model:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
df=pd.read_csv('diabetes.csv')
X = df.iloc[:,0:8] # feature matrix
y = df.iloc[:,8] # response vector
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.30, random_state=0)
knn = KNeighborsClassifier(n_neighbors=7,metric='euclidean')
knn.fit(X_train, y_train)
pred=knn.predict(X_test)
accuracy=accuracy_score(y_test,pred)
print('score: {}'.format(round(accuracy*100,2)))
# ==> 74.46%
```

18*Feature scaling example:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
df=pd.read_csv('diabetes.csv')
print(df.head())
df.isna().sum()
# create unscaled data where Blood pressure will dominate the glucose level
df['BloodPressure'] *= 9999999
df['Glucose'] /= 1000
X = df.iloc[:,0:8] # feature matrix
y = df.iloc[:,8] # response vector
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.30, random_state=0)
knn = KNeighborsClassifier(n_neighbors=7,metric='euclidean')
knn.fit(X_train, y_train)
```

```

pred=knn.predict(X_test)
accuracy=accuracy_score(y_test,pred)
print('score: {}'.format(round(accuracy*100,2)))
# ==> score: 62.34%
# with featutre scaling
from sklearn.preprocessing import StandardScaler
s = StandardScaler()
X_train=s.fit_transform (X_train)
X_test=s.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=7,metric='euclidean')
knn.fit(X_train, y_train)
pred=knn.predict(X_test)
accuracy=accuracy_score(y_test,pred)
print('score: {}'.format(round(accuracy*100,2)))
# ==> score: 74.46%

```

19*Types of Feature Scaling:

- Original Data
- StandardScaler
- MinmaxScaler

Save a model:

- Pickling comes as the standard module in Python which makes it easy to use for saving and restoring ML models.
- Pickle files can handle most Python objects including custom objects, making it a versatile way to save models.
- For small models, pickle approach is quite fast and efficient.
- When an ML model is unpickled, it is restored to its previous state, including any variables or configurations. This makes Python pickle files one of the best alternatives for saving ML models.

```

import pickle
# save the iris classification model as a pickle file
model_pkl_file = "iris_classifier_model.pkl"
with open(model_pkl_file, 'wb') as file:
    pickle.dump(model, file)
# load model from pickle file
with open(model_pkl_file, 'rb') as file:
    model = pickle.load(file)
# evaluate model
y_predict = model.predict(X_test)
# check results
print(classification_report(y_test, y_predict))

```

Chapter 08 : Decision Tree & Random Forest

01*Iris Species:

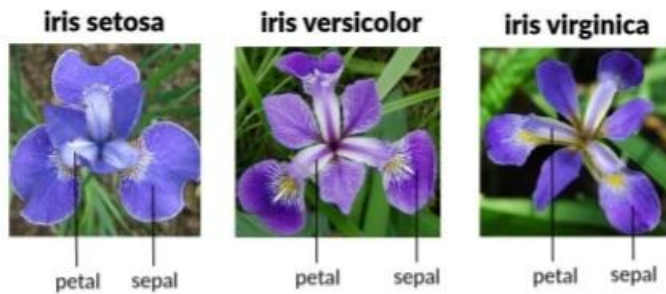
Iris Species (Classification)

Classify iris plants into three species in this classic dataset:

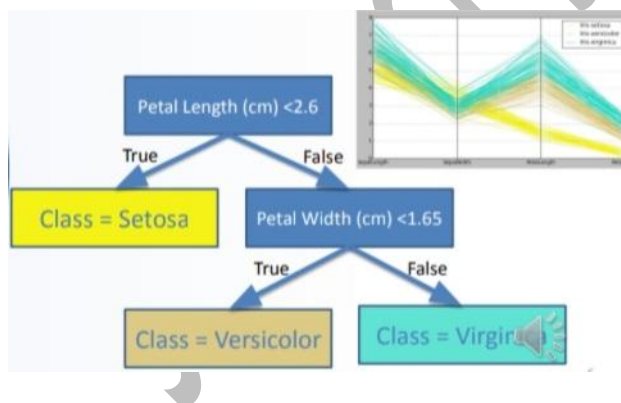
1-Setosa

2-Versicolor

3-Virginica



sepal-width	sepal-length	petal-width	petal-length
5.1	3.5	1.4	0.2
4.9	3	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5	3.6	1.4	0.2
7	3.2	4.7	1.4
6.4	3.2	4.5	1.5
6.9	3.1	4.9	1.5
5.5	2.3	4	1.3
6.5	2.8	4.6	1.5
6.3	3.3	6	2.5
5.8	2.7	5.1	1.9
7.1	3	5.9	2.1
6.3	2.9	5.6	1.8
6.5	3	5.8	2.2



02*Decision Tree Applications:

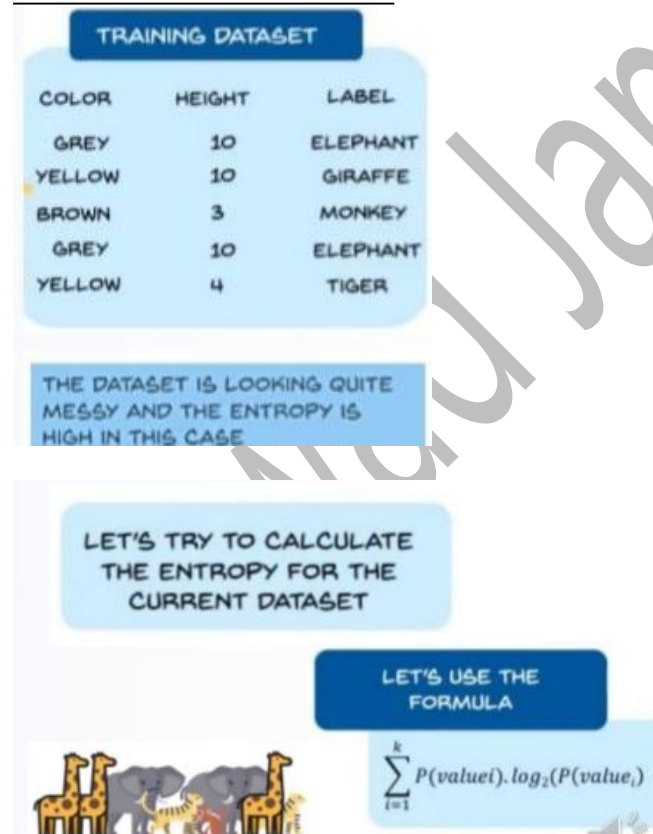
DTs are used in:

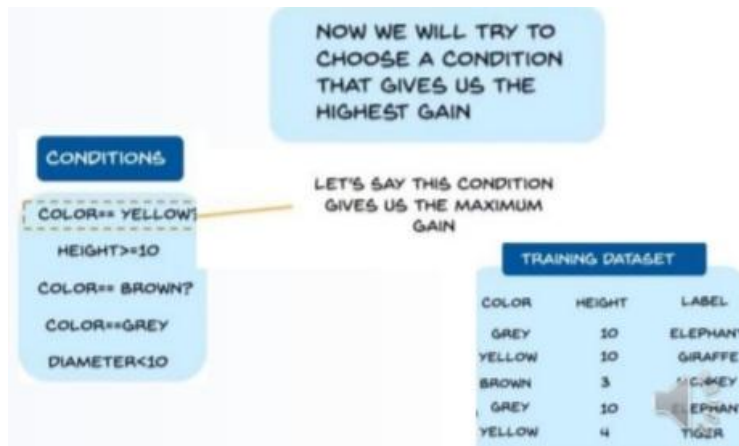
- marketing and sales,
- reducing churn rate, (customer stop using the product)
- anomaly and fraud detection,
- medical diagnosis, etc

03*Decision Tree:

- Is a supervised machine learning algorithm.
- Is one of the predictive modelling approaches used in Machine Learning.
- It can be used for both a classification problem as well as for regression problem.
- Has a Root node, Decision Node and Leaf nodes.
- Decision Trees (DTs) belong to supervised machine learning algorithms.
- The tree can be explained by two entities, namely decision nodes and leaves.
- The leaves are the final outcomes, and each node (decision nodes or internal nodes) within the tree represents a test on specific feature.
- These methods are used for regression as well as classification problems.
- Using the DT with a given set of inputs, one can map the various outcomes that are a result of the consequences or decisions.
- These trees are used to provide graphical outputs to the user based on several independent variables.
- DT is capable of handling heterogeneous as well as missing data.
- Classification can be performed without many computations.
- A DT is a tree-like graph.
- At each node, an edge is generated based on some specific characteristics of one of the attributes.
- We may well state that nodes represent a place where we pick an attribute and ask a question, and edges represent the answers to the question.
- Finally, the leaves represent actual output or class labels

How does a Decision Tree work:





04*Decision Tree Terminology:

- Root node: Root node is from where the DT starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.
- Leaf node: Leaf node is the final output node, and the tree cannot be segregated further after getting a leaf node.
- Splitting: Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- Branch/sub-tree: A tree formed by splitting the tree.

05*Entropy:

- Entropy is an information theory metric that measures the impurity or uncertainty in a group of observations.
- It determines how a decision tree chooses to split data.
- The image below gives a better description of the purity of a set.

06*Information Gain:

- We can define information gain as a measure of how much information a feature provides about a class.
 - Information gain helps to determine the order of attributes in the nodes of a decision tree.
- Information Gain = 1 – Entropy

07*Gini impurity Index:

Gini impurity index is a measure of the diversity of a dataset
 Gini index = probability of picking two distinct element
 Gini = 0 (Less diverse)
 Gini = 0.42 (Less diverse)
 Gini = 0.7 (More diverse)

$$\text{Gini Impurity} = 1 - \sum p_i^2$$

08*Max Tree Depth:

Max Depth: Controls the maximum depth of the tree that will be created.
 It can also be described as the length of the longest path from the tree root to a leaf.
 The root node is considered to have a depth of 0.
 The Max Depth value cannot exceed 30 on a 32-bit machine.

09*Decision tree Classifier:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
data = pd.read_csv('Week-08-1 Naive Bayes - US Voting.data',
names=['class', 'handicapped-infants', 'water-project-cost-sharing',
'adoption-of-the-budget-resolution', 'physician-fee-freeze',
'el-salvador-aid', 'religious-groups-in-schools',
'anti-satellite-test-ban', 'aid-to-nicaraguan-contras',
'mx-missile', 'immigration', 'synfuels-corporation-cutback',
'education-spending', 'superfund-right-to-sue', 'crime',
'duty-free-exports', 'export-administration-act-south-africa'])
data_x = data.iloc[:,1:17]
data_y = data['class']
data_x = pd.get_dummies(data_x)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(data_x, data_y, test_size=0.2)
# Train the DT classifier
from sklearn.tree import DecisionTreeClassifier
model=DecisionTreeClassifier(criterion='entropy',random_state=44)
model.fit(X_train, y_train)
y_predict = model.predict(X_test)
# Calculate the accuracy of the model
from sklearn.metrics import accuracy_score
print ( accuracy_score(y_test,y_predict))
# Plot the results
from sklearn.tree import plot_tree
plt.figure(figsize=(6,6), dpi=150)
plot_tree(model, feature_names=list(data_x.columns),max_depth=1)
plt.show()
```

10*Decision Tree And Mean Square:

- In Decision Trees for Classification, we saw how the tree asks right questions at the right node in order to give accurate and efficient classifications.
- The way this is done in Classification Trees is by using 2 measures , namely Entropy and Information Gain.
- But with regression since we are predicting continuous variables, we cannot calculate the entropy and go through the same process.
- We need a different measure now.
- A measure that tells us how much our predictions deviate from the original target and that's the entry-point of mean square error.

11*Decision Tree/ Variance:

- Variance is a method for splitting the node used when the target variable is continuous, i.e., regression problems .
- It is so-called because it uses variance as a measure for deciding the feature on which node is split into child nodes.
- It quantifies how much the values differ from the mean (average) of the dataset.
- In decision tree regression, variance is used to assess the homogeneity of the target variable within subsets of the data.
- Lower variance indicates that the values are closer to the mean and, therefore, more homogeneous.

12*Decision Tree regressor:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score
df=pd.read_csv('50_Startups.csv')
df = pd.get_dummies(df)
X = df.drop(['Profit'],axis=1) # feature matrix
y = df['Profit'] # response vector
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,
random_state=0)
#Training the model
regressor=DecisionTreeRegressor(random_state=44,criterion='squared_error')
regressor.fit(X_train, y_train)
y_pred=regressor.predict(X_test) #make prediction on test set
score=r2_score(y_test,y_pred)
print(score)
from sklearn.tree import plot_tree
plt.figure(figsize=(5,4), dpi=150)
plot_tree(regressor, feature_names=list(X.columns),max_depth=2)
plt.show()
```

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

13*Decision Tree Advantages:

- Decision Trees are not sensitive to the scale of the input variables, meaning they do not require the data to be normalized or standardized before training.
- They can handle both continuous and categorical variables, and are able to identify patterns in the data regardless of the scale of the input variables.
- This can be an advantage when working with data that has different units of measurement or when the data is not normally distributed.
- It saves data preparation time, as they are not sensitive to missing values and outliers.

14*Decision Tree Disadvantages:

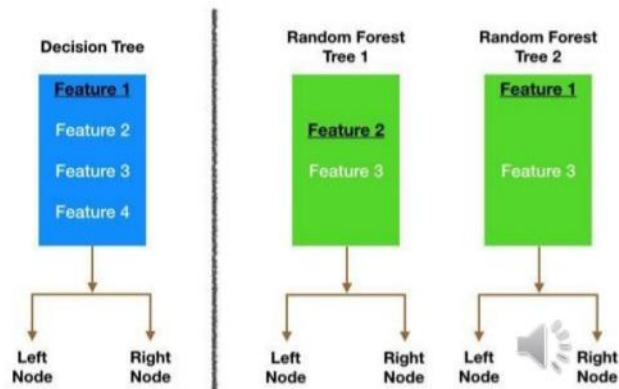
- More number of decisions in a tree, less is the accuracy of any expected outcome.
- DTs are easy to use when compared to other decision-making models but creating large DTs that contain several branches is a complex and time-consuming task.
- Unconstrained, individual trees are prone to over-fitting, but this can be alleviated by ensemble methods.

15*Random Forest:

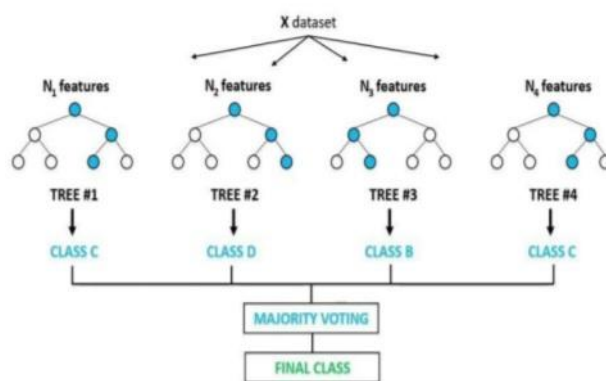
- Random forest is a Supervised Machine Learning Algorithm that is used widely in Classification and Regression problems.
- It builds decision trees on different samples and takes their majority vote for classification and average in case of regression.

16*Random Forest is a classifier that contains:

A number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset.



Random Forest Classifier



17*Why use Random Forest?

- ☐ It takes less training time.
- ☐ It predicts output with high accuracy, even for the large dataset it runs efficiently.
- ☐ It can also maintain accuracy when a large proportion of data is missing.

18*Random Forest steps:

- ☐ Step-1: Select random K data points from the training set.
- ☐ Step-2: Build the decision trees associated with the selected data points (Subsets).
- ☐ Step-3: Choose the number N for decision trees that you want to build.
- ☐ Step-4: Repeat Step 1 & 2.
- ☐ Step-5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

19*Random Forest Applications:

- ☐ Banking: for the identification of loan risk.
- ☐ Medicine: disease trends and risks of the disease can be identified.
- ☐ Land Use: We can identify the areas of similar land.
- ☐ Marketing: Marketing trends can be identified.

20*Random Forest Classifier:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
data = pd.read_csv('Week-08-1 Naive Bayes - US Voting.data',
names=['class', 'handicapped-infants', 'water-project-cost-sharing',
'adoption-of-the-budget-resolution', 'physician-fee-freeze',
'el-salvador-aid', 'religious-groups-in-schools',
'anti-satellite-test-ban', 'aid-to-nicaraguan-contras',
'mx-missile', 'immigration', 'synfuels-corporation-cutback',
'education-spending', 'superfund-right-to-sue', 'crime',
'duty-free-exports', 'export-administration-act-south-africa'])
print(data.head(5))
data_x = data.iloc[:,1:17]
data_y = data['class']
data_x = pd.get_dummies(data_x)
X_train, X_test, y_train, y_test = train_test_split(data_x, data_y, test_size=0.2)
model=RandomForestClassifier()
model.fit(X_train,y_train)
y_predict = model.predict(X_test)
from sklearn.metrics import confusion_matrix
print ( confusion_matrix(y_test, y_predict) )
from sklearn.metrics import accuracy_score
print ( accuracy_score(y_test,y_predict))
from sklearn.metrics import classification_report
print(classification_report(y_test, y_predict))
```

21*Random Forest Regression:

```
import pandas as pd
df=pd.read_csv("ford.csv")
print(df.head())
print(df.shape)
print(df.isnull().sum())
df=pd.get_dummies(df)
print(df.shape)
X=df.drop(['price'],axis=1)
y=df['price']
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
from sklearn.ensemble import RandomForestRegressor
model=RandomForestRegressor()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
from sklearn.metrics import r2_score
score=r2_score(y_test,y_pred)
print(score)
```

22*Decision Tree VS Random Forest:

Decision Trees

1. Decision trees normally suffer from the problem of overfitting if it's allowed to grow without any control.
2. A single decision tree is faster in computation.
3. When a data set with features is taken as input by a decision tree it will formulate some set of rules to do prediction.

Random Forest

1. Random forests are created from subsets of data and the final output is based on average or majority ranking and hence the problem of overfitting is taken care of.
2. It is comparatively slower.
3. Random forest randomly selects observations, builds a decision tree and the average result is taken. It doesn't use any set of formulas.

23*Random forest and features:

- Random forests consist of 4 –12 hundred decision trees, each of them built over a random extraction of the observations from the dataset and a random extraction of the features.
- Not every tree sees all the features or all the observations, and this guarantees that the trees are de-correlated and therefore less prone to over-fitting.
- Each tree is also a sequence of yes-no questions based on a single or combination of features.
- At each node (this is at each question), the tree divides the dataset into 2 buckets, each of them hosting observations that are more similar among themselves and different from the ones in the other bucket.
- Therefore, the importance of each feature is derived from how "pure" each of the buckets is.

24*Does it work differently for Classification and Regression?

- For classification, the measure of impurity is either the Gini impurity or the information gain/entropy.
- For regression the measure of impurity is variance.
- Therefore, when training a tree, it is possible to compute how much each feature decreases the impurity.
- The more a feature decreases the impurity, the more important the feature is.
- In random forests, the impurity decrease from each feature can be averaged across trees to determine the final importance of the variable.

25*Overfitting and Random Forest:

- Instead of relying on one DT, the Random Forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.
- The greater number of trees in the forest leads to higher accuracy and prevents the problem of over-fitting.
- Each tree draws a random sample from the original dataset when generating its splits, adding a further element of randomness that prevents over-fitting

26*Advantages:

- The method has not encountered over-fitting problem.
- The method can be used for both classification and regression tasks.
- Relatively the method requires very little preprocessing of data.
- Generally, the method maintains accuracy in the presence of missing data and is robust to outliers.
- The method has an implicit feature selection as it gives estimates on what variables are important.

27*Disadvantages:

- The method is easy to use but analyzing it is difficult.
- When the method generates large number of trees, then it can slowdown making real-time predictions.

Chapter 09: K-Means

01*K-Means: Clustering:

KMeans is a clustering algorithm that assign groups to a dataset without the need for previously labelled data.

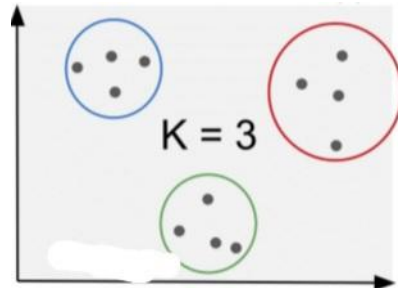
- Unsupervised Machine Learning technique,
- A clustering algorithm,
- A grouping of data points; Classify each data point into a specific group (called a cluster),
 - Data points that are in the same group should have similar properties (in some sense),
 - Data points in different groups should have dissimilar properties,

What is a good real-life example of KMeans Clustering?

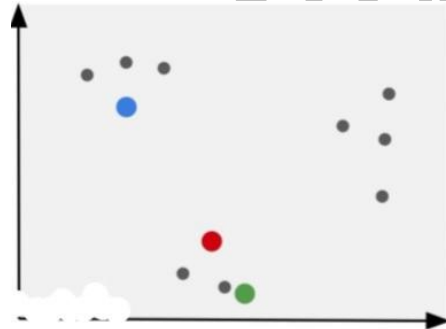
- Document clustering in Information Retrieval
- Customer segmentation
- Recommender systems on web platforms
- Pattern recognition.
- In Marketing to improve user experience or draw insights from data.
- The algorithm iteratively divides data points into K clusters by minimizing the variance in each cluster.
- First, each data point is randomly assigned to one of the K clusters.
- Then, we compute the centroid (the center) of each cluster, and reassign each data point to the cluster with the closest centroid.
- We repeat this process until the cluster assignments for each data point are no longer changing.

02*K-Means Clustering Algo:

1.Select the number of clusters (K)In KMeans clustering, the number of clusters (K) needs to be defined.This is the "K" in "K-Means clustering".

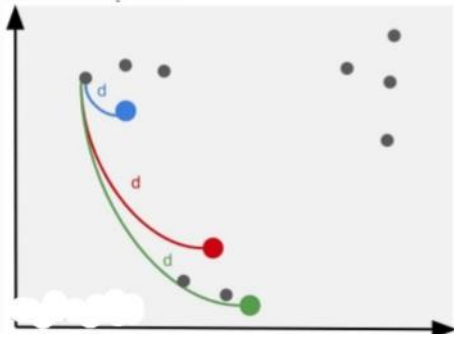


2.Randomly select a number of data points that matches the number of clusters.These will become the initial clusters (cluster centroids).

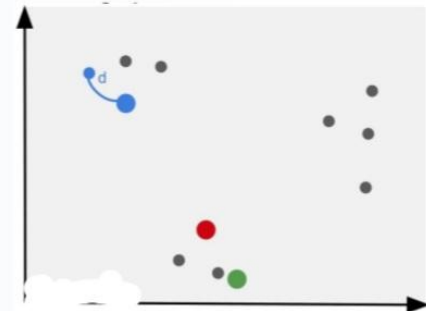


3. Measure the distances between each point to its initial cluster

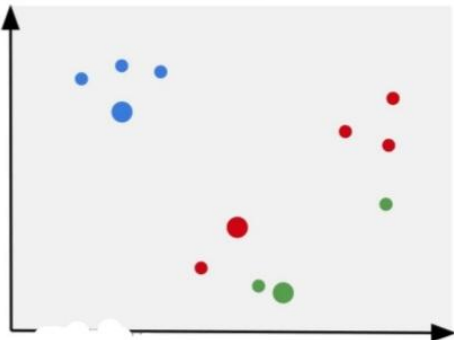
The clustering algorithm calculates the distances between each point to each cluster point.



4. Assign each datapoint to its nearest initial cluster Assign the first point to the closest initial cluster.



5. Repeat the calculations for each point

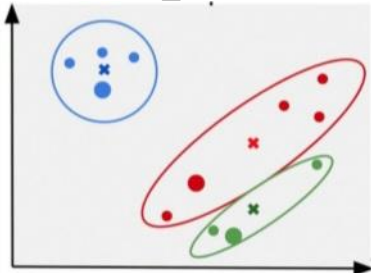


6. Calculate the mean of each cluster

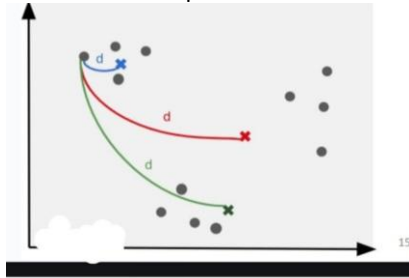
The next step is to add all the data points in each cluster and divide them by the number of points in the cluster, essentially calculating the mean of the cluster.

7. Assign the mean as the new cluster centroid

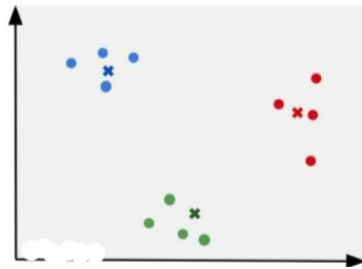
Then, the mean is selected as the next cluster centroid.



8. Measure each point to the new cluster centroid

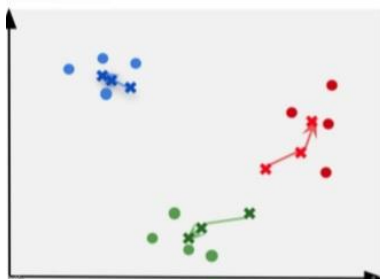


9. Redefine clusters and assign the new mean as the next cluster centroid



10. Repeat process until convergence

Repeat the process until the clusters do not change anymore.



11. Select the parameters with the smallest SSE

The goal of machine learning clustering algorithms is to find clusters that minimize errors. In the case of KMeans, the error is often calculated with the Sum of Squares Errors (or SSE).

03*Best K value:

How to use the right number of clusters (K)?

The optimal number of clusters generally equals the number of classes.

If you know that number of classes, use it as your number of clusters.

For example, if you know that you are clustering webpages into 5 different categories, you can use 5 as your number of clusters. Otherwise, you can use the Elbow method.

04*The Elbow Method:

The elbow method is used in cluster analysis to help determine the optimal number of clusters in a dataset.

It works by:

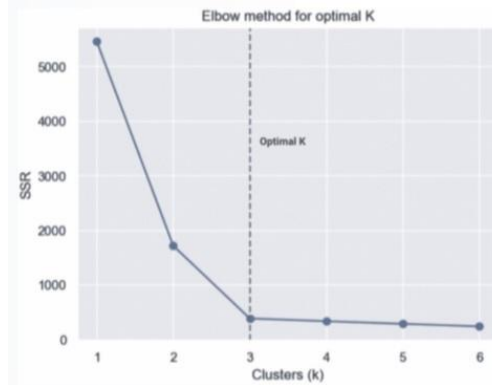
- ☐ Defining a range of K values to run K-Means clustering on
- ☐ Evaluating the Sum of Squares Errors (SSE) for the model using each of the defined numbers of clusters.

The optimal K value is usually found at the “elbow” where the curve starts to become more constant.

Rules of the Elbow Method:

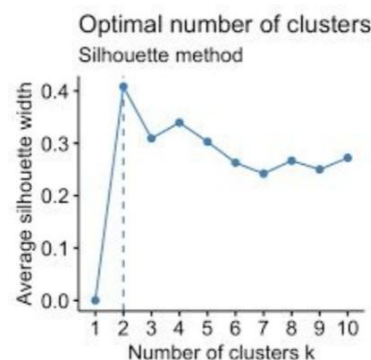
- The maximum of clusters you can have is equal to the number of individual points.
- When the number of clusters equals the number of points, the distortion (sum of squared distances) equals 0.
- More clusters involve more computation costs. You can also choose the best K value programmatically or by using the silhouette coefficient, or alternatively by using gap statistics.

Elbow curve:



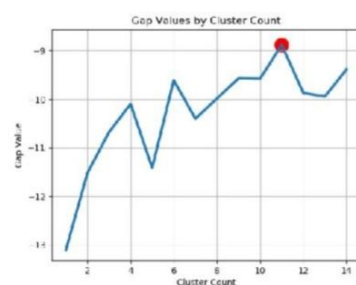
05*Silhouette Coefficient:

- Silhouette analysis can be utilized to examine the amount of separation between the clusters.
- The silhouette plot shows how close each point in a cluster is to points in the neighboring clusters and thus provides a way to visually determine parameters such as the number of clusters.



06*Gap statistic:

- In a distribution graph, a gap is an interval which contains no data;
- on the other hand, a peak is the highest point of a data set.
- When several data points lie close together, we say there is a cluster, which typically means that these data points have similar characteristics



07*KMeans Inertia (SSE):

KMeans Inertia (Sum of Squares Errors = SSE):

- Calculates the sum of the distances of all points within a cluster from the centroid of the point.
- It's a formula used to measure the difference between the data obtained by the prediction model that has been done previously.
- SSE is often used as a research reference in determining optimal clusters. The sum of squares errors will often take different names:

☐ Inertia (Scikit-learn)

☐ Distortion (SciPy)

$$\sum_{i=1}^n (y_i - \bar{y})^2$$

08*When to use KMeans?

KMeans is best suited for:

- ☐ Unsupervised learning problems
- ☐ With numeric data, does not work on categorical data
- ☐ Better for linear datasets

Advantages:

- ☐ Simple and fast computation

Disadvantages:

- ☐ Need to define the clusters
- ☐ Hard time handling outliers
- ☐ Fail on non-linear datasets

09*KMeans with Scikit-learn:

- ☐ Train and cluster data using KMeans
- ☐ Plot the clustered data
- ☐ Pick the best value for K using the Elbow method.

10*KMeans Example with Python

- Customer segmentation Dataset:

CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

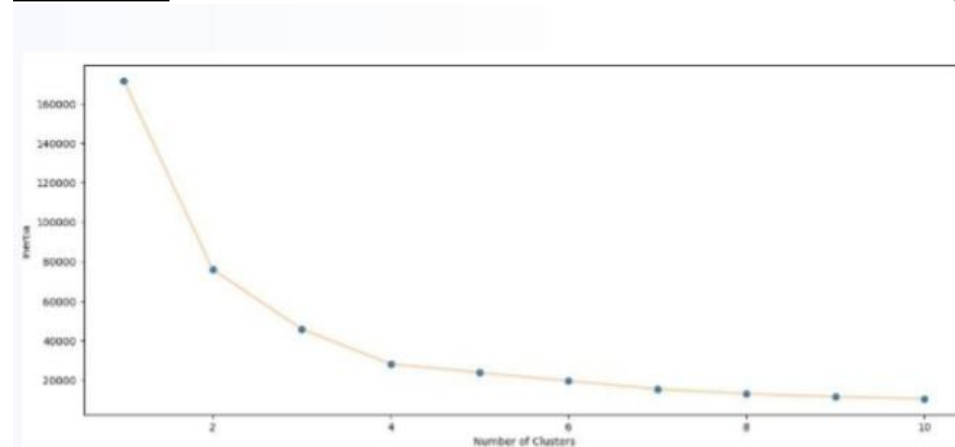
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
df = pd.read_csv('Mall_Customers.csv')
df.head()
df.shape
df.describe()
df.dtypes
df.isnull().sum()
'''Segmentation using Age and spending Score'''
X1 = df[['Age', 'Spending Score (1-100)']].iloc[:, :].values
```

```

inertia = []
for n in range(1, 11):
    algorithm = (KMeans(n_clusters = n, init='k-means++', n_init = 10,
        max_iter=300, tol=0.0001, random_state= 111, algorithm='elkan'))
    algorithm.fit(X1)
    inertia.append(algorithm.inertia_)
plt.figure(1, figsize = (15, 6))
plt.plot(np.arange(1, 11), inertia, 'o')
plt.plot(np.arange(1, 11), inertia, '-', alpha = 0.5)
plt.xlabel('Number of Clusters'), plt.ylabel('Inertia')
plt.show()

```

Elbow Curve:



```

algorithm = (KMeans(n_clusters = 4, init='k-means++', n_init = 10, max_iter=300, tol=0.0001, random_state= 111, algorithm='elkan'))
algorithm.fit(X1)
labels1 = algorithm.labels_
centroids1 = algorithm.cluster_centers_
import matplotlib.pyplot as plt
plt.style.use('default')
X = X1[:,0]
y = X1[:,1]
plt.scatter(X, y, c=algorithm.labels_, alpha=0.8)
cluster_centers = algorithm.cluster_centers_
cs_x = cluster_centers[:,0]
cs_y = cluster_centers[:,1]
plt.scatter(cs_x, cs_y, marker='*', s=100, c='r')
plt.title('KMeans')
plt.xlabel('Age'), plt.ylabel('Spending Score')
plt.show()

```